

Seq

A Hierarchical and Modular MIDI Sequencer

Sean Luke

Department of Computer Science
George Mason University
sean@cs.gmu.edu

Seq Version 10

Manual Version 9.0

January 2026

Where to Obtain Seq

<http://github.com/eclab/seq/>

Copyright 2024–2026 by Sean Luke.

Thanks to Filippo Canovalini and Cesar Liendo.

Get the latest version of this document or suggest improvements here:

<http://github.com/eclab/seq/>

This document is licensed under the **Creative Commons Attribution-No Derivative Works 3.0 United States License**, except for those portions of the work licensed differently as described in the next section. To view a copy of this license, visit <http://creativecommons.org/licenses/by-nd/3.0/us/> or send a letter to Creative Commons, 171 Second Street, Suite 300, San Francisco, California, 94105, USA. A quick license summary:

- You are free to redistribute this document.
- **You may not** modify, transform, translate, or build upon the document except for personal use.
- You must maintain the author’s attribution with the document at all times.
- You may not use the attribution to imply that the author endorses you or your document use.

This summary is just informational: if there is any conflict in interpretation between the summary and the actual license, the actual license always takes precedence.

Contents

0	Introduction	2
0.1	Where We're Going with Seq	4
1	The User Interface	5
1.1	The Motif List (and the Edit, Add, Motif, and File Menus)	6
1.2	The Transport (and the MIDI Menu)	7
1.3	The Current Motif Display (and the Options and Per-Motif Menus)	8
1.4	The Inspectors and Text Notes	9
2	The Motifs	10
2.1	Notes	11
2.2	Step Sequence	16
2.3	Silence	19
2.4	Series	20
2.5	Parallel	23
2.6	Automaton	26
2.7	Select	29
2.8	Arpeggio	32
2.9	Filter	35
2.10	Modulation	40
3	Macros and Macro Children	44
4	Parameterization	45
4.1	Parameterized Control of MIDI Data in Series	48
4.2	Select's Overriding of Parameters	49
5	Developing for Seq	49
5.1	Model and Dynamic Runtime Overview	49
5.2	Graphical Interface Overview	51
5.3	Common GUI Widgets	52
5.4	Building a Module	53
5.5	Hints	53

0 Introduction

Seq is a music sequencer quite different from other sequencers. Rather than lay out MIDI clips in a linear timeline as you would in a DAW, or play small iterations of patterns like a step sequencer, Seq groups chunks of music into sequences, then groups multiple sequences into even larger sequences, and so on, until it has formed a large sequence, essentially a program describing how its elements are to be played. Thus Seq is creates a sequence in the form of a *hierarchy* of subsequences.

Seq goes further than this, however: different sequences can be built out of the same subsequences. Thus subsequences may be thought of as *modular*, and they are reused in different guises and in different scenarios.

Music made with Seq tends to be composed bottom-up: you start with some basic building blocks (step sequences, small MIDI clips), combine them in different ways, and then combine the combinations, and so on, building up larger pieces of compositions until you have your finished song. Seq can also be useful for real-time performance, as you can modify the structure and parameters of a song in real-time. Seq even has a dedicated object, called *Select*, which lets you launch MIDI subsequences just like a clip launcher in Ableton etc. Seq sequences, or pieces of them, can also be broken out and easily shared in other compositions. Finally, many Seq features can be *parameterized*, meaning that you can change them automatically or manually at a very abstract level.

Seq was largely written from 2023–2025 by Sean Luke with help from Filippo Carnovalini.

Who is Seq For? Seq was developed for research purposes: a major goal is to provide an environment where we could insert modules that automatically generate music or automatically filter and modify it. We also wanted an architecture that was easily modified, so for example an AI optimizer might rip apart an entire song and restructure it into a different form. Seq doesn't have those kinds of extensions in its public distribution, but stay tuned.

So if you're not an academic, while use Seq? Seq's approach to building songs is very unusual. It has disadvantages to be sure: compared to a linear arranger in a DAW, it doesn't at present show clips side-by-side in a timeline, so you have to do some math in your head. But Seq makes it easy to reuse and modify pieces of music in unusual ways to form a whole composition: and Seq has significant performance facilities. We think Seq would be of particular interest to modular synthesizer composers and musicians, to composers and producers of ambient music, EDM, IDM, electronica, long-form music, minimalist music, and other music involving high degrees of experimentation, or reuse and remixing. Though it's computer software, Seq might be of significant interest to DAWless setups. But in fact we use Seq for all sorts of music.

Seq is Hierarchical By **hierarchical** we mean that a Seq sequence is composed of a *hierarchy* of sequence objects, each of which are groupings of other sequence objects (which themselves can be groupings of sequence objects, and so on). You start playing one object, and as it runs, it starts playing its child objects according to some rules. They in turn start playing their children, and so on.

An example of a trivial hierarchical sequencer is **song mode**, commonly found in drum machines and grooveboxes. The earliest music sequencers just output a simple looped pattern of perhaps of eight or sixteen notes, voltages, or drum triggers. But with the advent of Roger Linn's **LM-1** drum machine he introduced the notion of what he called a **chain** of patterns. In the LM-1, you programmed multiple patterns, then defined a set of instructions (the chain) that said, for example, "First play pattern A three times. Then Play Pattern B once. Then play Pattern A twice. Then play pattern C once." A pattern could be played on its own, or you could play the chain. The patterns were basic sequence objects, and the chain was was their parent in a simple two-level hierarchy. Chains eventually became known as **songs**, and playing a chain eventually became popularized in drum machines as putting the machine in so-called "song mode."

Now imagine: what if you had a **chain of chains**? Perhaps you might have a chain X that said "Play all of Chain Y three times, then play Pattern B twice, then play Chain Z". You could create a larger and larger hierarchy of chains.

Parents don't just have to be chains (or in Seq's terminology, **Series** objects). You could instead have a parent that says "Play Chain X and Chain Y in parallel, with Chain Y coming in delayed by 5 beats." Or you

could have a parent that says “Play either Chain X or Chain Y, selected at Random”. And so on. You could create a complex hierarchy of parent objects.

The children at the very bottom don’t have to be just simple patterns either. They can be entire chunks of MIDI, or melody generators, or whatnot.

Furthermore, when in the course of playing a child generates MIDI, the MIDI is sent to its parents (and their parents, and so on), and they have a chance to modify it before it is finally emitted.

Seq is Modular By modularity we mean that not only can a parent object in the hierarchy have multiple children, but a given object in the hierarchy can also be shared by multiple parents. Modularity enables **reuse** of objects, and perhaps transforming them or creating variations. Modularity also allows sequence hierarchies to be **compact**.

As a simple example, imagine if Chain X had Chain Y as a child as well as Pattern A. But Chain Y *also* had Pattern A as a child, along with Pattern B. Both Chain X and Chain Y use Pattern A, though perhaps at different times and in different ways. You could also imagine a Chain Z which used by Chain X and Chain Y, and thus Chain Y was also being used twice, in different ways.

Figure 0 at right shows this very arrangement of these chains and patterns. In the computer science world, this kind of structure is known as a *Directed Acyclic Graph* or *DAG*. It’s a *graph* because every object is connected to other objects with lines called *edges*. It is *directed* because there is a clear relationship between parents and children: that is, the edges are arrows indicating who the child is. It’s *acyclic* because no child can be its own ancestor: you can’t have loops (or “cycles”). That is, all edges are pointing *down*.

You don’t have to start playing at Chain Z. You could instead choose to start playing at (say) Chain Y, and so Z and X would both just be ignored. Seq calls the node that you’re starting playing at the **Root** of the current sequence (even if it itself has parents).

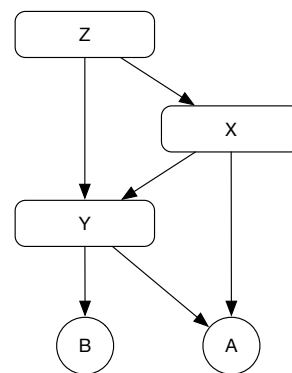


Figure 0 A Simple Directed Acyclic Graph (DAG)

Some Terminology Seq’s sequences are DAGs built out of basic musical building blocks (step sequences, chunks of MIDI, etc.), and objects which group other building blocks into subsequences (including grouping other subsequences together). All these objects, both basic and grouping, are known as **motifs**. Thus (to use the LM-1’s terminology) both individual step sequence patterns and pattern chains are represented as kinds of motifs. One motif is designated the **root motif**, and is the start point when playing. Motifs which group other motifs are known as the **parents** of those **children**.

When you’re editing a Motif, you’ll be shown the **Motif Display** and, to its right, various **Inspectors** to modify the features of the Motif and how it manages its children (if it has them). All your Motifs are collected into one group, called the **Motif List**.

Macros and Parameterization Seq has two more powerful tricks up its sleeve, but neither of them is necessary to understand in order to use Seq: thus they are discussed at the end of the manual.

First, you can encapsulate an entire sequence into a single motif called a **Macro**. This allows you to use one sequence as an element to be called from another sequence. Macros can also be set up to have their own children, so the encapsulated sequence can call motifs outside, in the outer sequence: this allows you to create Macros which are essentially pattern templates. For more information on Macros, see Section 3.

Second, a great many of the features in motifs are **parameterizable**. This means you can attach them to **parameters** in the motif itself, which can in turn be changed by the motif’s parents or ancestors. This allows a parent to automatically change a variety of features in its children or descendants. For example, a parent could use a low-frequency envelope to change the pitch of certain notes in one of its children. Parameters can also be changed with Control Change (CC) messages from external controllers, and parameters can also be used to send out many different kinds of MIDI messages, including CC, NRPN, and so on. Parameterization is a complex subject: for more information, see Section 4.

0.1 Where We're Going with Seq

Seq's sequence structure is very amenable to customization and automated modification. Our ultimate goal is to ask what kinds of machine learning, optimization, or search techniques we could add to Seq to convert it into a **co-creative tool** for the composer, that is, a tool that works with the composer, bouncing ideas back and forth, to produce a song. We have previous experience in tools of this sort (see Sean Luke's **Edisyn** for example). To this end, we'll be adding motifs which use AI or other tools to generate automated music, and also motifs which take music and modify it automatically in interesting ways.

We'd also like to see a Live Coding module for Seq, though there are some challenges due to the poor options among Live Coding programming languages.

In the immediate future, we'll be adding external sync, a dark mode, interface improvements, and more MIDI file generation.

1 The User Interface

Seq is organized with a single window, shown in Figure 1. At top left is the **Transport**, which contains the record, play, pause, stop, and loop buttons, plus the current time clock in ticks, beats, measures, and parts. Below the transport are two expandable regions for **Clock Options** and for **Root Arguments**.

Below the Transport is the **Motif List**. This is a scrollable list of every motif you have added to your sequence. Below the Motif List are the **Motif Editing Buttons** to add, remove, and duplicate motifs in the list; as well as the **Motif Stack Buttons**, which let you return to previously visited motifs in order.

In the center of the window is the **Current Motif Display**. This shows the selected motif and lets you edit it. Different motifs have different displays. The one shown in Figure 1 is a Step Sequence. Below the Current Motif Display are the **Motif Display Editing Buttons**, which let you add to the display, delete from it, and copy elements in it; and the **Display Zoom Buttons** which let you zoom in and out. Different motifs have different options for these buttons.

At the right are the **Inspectors**. These are regions for editing the parameters of the current motif itself, and various elements in the motif. At top right is the **Current Motif Inspector**. This holds editable features of the current motif. For the Step Sequence, editor features include things such as the sequence name, length, degree of swing, etc.

Below the Current Motif Inspector are the **Motif Element Inspectors**. Different motifs have different kinds of elements, so a variety of things may appear here. In this example, there is an inspector for the features of the currently selected Track (Track 3—notice this track's number is highlighted in red). Below it is an inspector for the features of the currently selected step in this track (Step 7 of Track 3).

Below the Inspectors is a resizable region for **Text Notes** for each Motif.

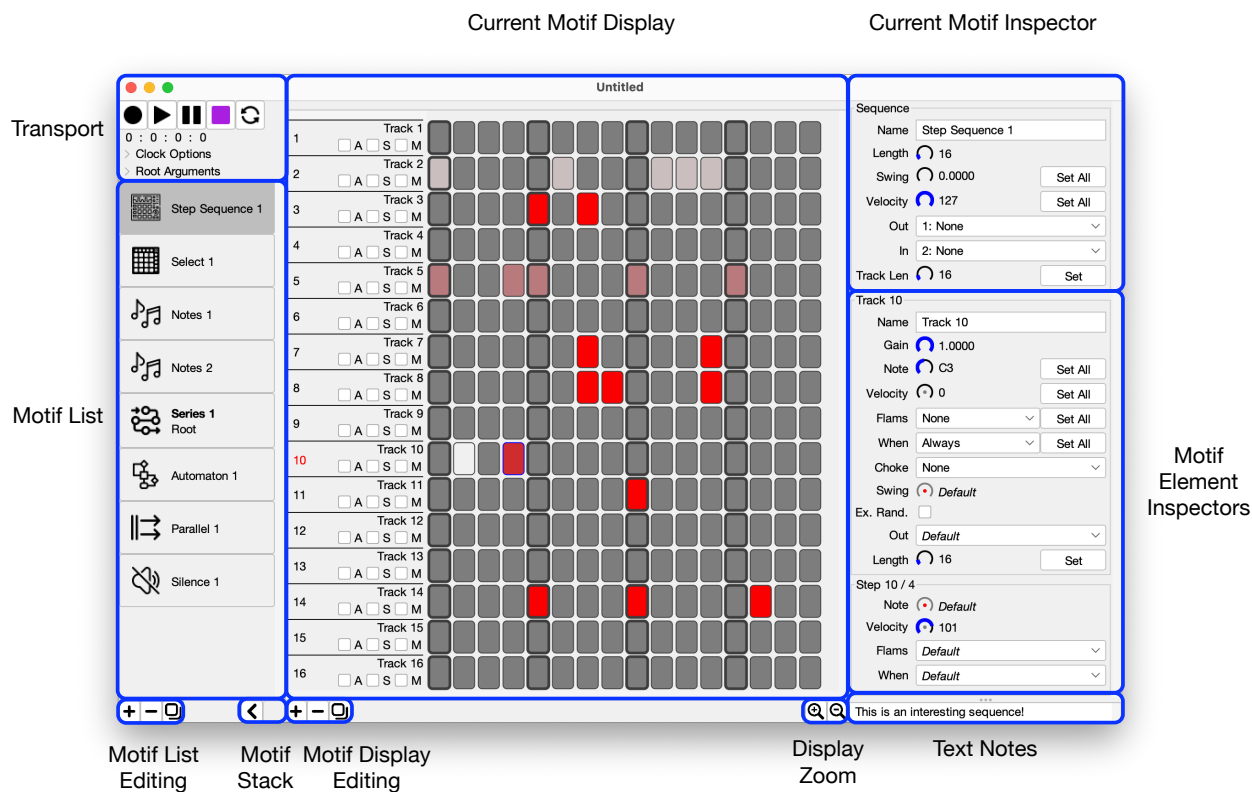


Figure 1 Seq graphical interface elements.

Dials Most widgets are pretty standard in Seq. But **dials** require a little explanation. A dial (usually) sets a numerical value: you set it by clicking on the dial proper and dragging up or down. The dial then shows the numerical value to its right. For example, the *Velocity* dial at right is set to 117.

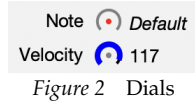


Figure 2 Dials

Some dials can be set to **special values**. These dials will have a little gray dot in the center of them (as was the case for the Velocity dial). Sometimes there is only one special value. If you double-click on the dial, it will automatically be set to this special value. If there are more than one special value, when you double-click on the dial, a menu will pop up to let you choose the special value you want. If a special value is set, it is shown in *italics* and the little gray dot turns red. For example, the *Note* dial has been set to the special value *Default*.

If a dial has no special values available, no dot will appear.

1.1 The Motif List (and the Edit, Add, Motif, and File Menus)

Each motif has a slot in the **Motif List**, where it is shown with its current name. You can rearrange the order of the motifs and select a new motif to display by clicking on it. This will cause it to be displayed in the **Current Motif Display**. One motif is designated the **Root Motif**, with the word **Root** under the motif name. This is the start point that will be played when the sequence is played.

When you select a motif, other motifs may appear in light blue. These are the **parents** of the motif, that is, the motifs which include the motif among their children. Others appear in pink: these are the **children** of the motif.

When a motif is currently playing, its text will be highlighted in red. Since a currently playing motif may also be playing its children, multiple motifs may appear in red at the same time.

If a motif is armed for recording, it will appear with a little red circle before it.

Below the Motif List are the **Motif List Editing Buttons**, which let you add new motifs to the list, delete them, and copy them. Most importantly, clicking the “+” button will pop up the a **List of Available Motifs**, as shown at right in Figure 3. Select a motif and a new one will appear in the Motif List. You can also choose the same motifs from the **Add Menu**.

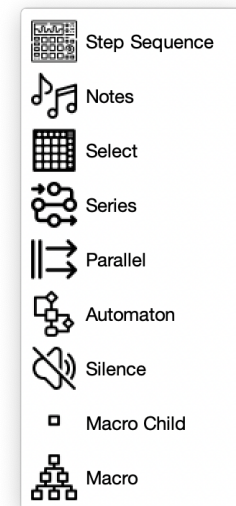


Figure 3 Available Motifs

Also below the Motif List are the **Motif Stack Buttons**. When you select a motif, the current motif display changes to reflect the newly selected motif. The previously selected motif is pushed on the stack. You can go back to previous motifs by pressing the “<” button, and return to later selected motifs with the “>” button. These buttons only appear if there is a motif to go back (or forward) to.

Motif Menu The Motif Menu has several options.

- You can **set the Root Motif** to the currently selected motif.
- You can **automatically set** motifs to be the the Root Motif whenever you select them.
- You can **Sort Motifs**, which organizes motifs so that children appear lower in the list than their parents.
- You specify which kind of Motif is the **initial one displayed** when Seq launches.
- You can change the motif buttons to a **smaller size**.
- You can set things such that **arming a motif** automatically disarms all the others. If your Motif (notably Notes) is set to automatically arm itself when first created, this will also disarm the others.
- You can **disarm all motifs** so that they no longer record MIDI. This is useful if you’ve forgotten which ones you’ve armed so far.

File Menu The File Menu allows you to load and save sequences and to clear the existing sequence and restart with a new one. You can also **Export** the sequence starting at the Root Motif, discarding all motifs that are not its children nor descendants. Finally, you can **Merge** another sequence into the current one: this adds to the current sequence copies of all the Motifs in the other sequence.

Seq can also load Type 1 and Type 2 MIDI files via **Load MIDI File....** If the MIDI file has multiple tracks, or a single track with multiple channels, Seq will load the file as multiple Notes Motifs, with a Parallel Motif as their common parent. Otherwise, it will load the file into a single Notes Motif. We will discuss the Notes and Parallel Motifs later.

Edit Menu The Edit Menu largely concerns itself with **Undo** and **Redo**. However Undo works differently than you're used to: only structural changes are undone. If you change a parameter, it's not undoable. However you can add an undo point prior to the parameter change by choosing **Checkpoint** before making the change.

1.2 The Transport (and the MIDI Menu)

The Transport is how you play the sequence or start recording incoming MIDI. The transport has a Record, Play, Pause, and Stop Button, as well as a Loop Toggle. Buttons are highlighted in colors to indicate the current playing state. Press the **Play** button to start playing, and press **Stop** to stop playing. Press the **Record** button to start recording. The **Loop Toggle** loops when playing: when the sequence has ended, it will start playing again.

The **Fast Forward** button allows you to jump forward in time. When you press it, Seq will ask you what time to fast forward to, and then will play the entire sequence at high speed (not outputting any MIDI) until Seq reaches the desired timestamp.¹ At that point Seq will *pause* the sequence and the Fast Forward button will turn into a **Pause** toggle button. When you press the Pause button to resume playing the sequence, Seq will continue playing normally.

The **Pause** button also is available to pause or resume the sequence during normal playing or recording.

When you start playing the sequence, play commences with the **Root Motif**.

If the Root has parent motifs, they are ignored and not played.

Recording only has an effect on certain Motifs, such as the **Notes** motif, when they are armed. Otherwise it acts just like playing for all other Motifs.

Below the Transport buttons is the **Clock**, which shows the current play time in (right to left) ticks (192 ticks per beat), beats, measures, and parts (256 measures per part). To the left of the clock is a small dot. This dot will light up momentarily when Seq receives MIDI information (other than Note Off) from any of its currently registered devices and channels.

Below the clock are two expandable regions: the **Clock Options** and the **Root Arguments**. The Clock Options include the tempo, whether the clock is Internal or External, the number of beats per measure, whether the metronome plays for recording, for recording and playing, or for none; whether a count-in is provided for recording, for recording and playing, or for none; and the metronome volume and pitch. Three settings of some of these elements are stored with your Sequence; others persist in the application's preferences.

We'll discuss the Root Arguments later in Section 4. However we mention one item here: the **Random Seed**. This is the seed of the random number generator used by Seq when playing, and the generator is always reset to that seed each time we play again. That way the sequence will always be deterministic. The seed is saved with the Sequence when you write it to a file, but new Sequences get new random seeds. You can always change this seed to cause the sequence's random choices to be different.

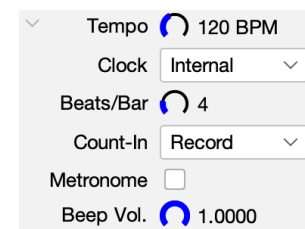


Figure 4 Clock Options

¹In a normal sequencer, you can just jump to a spot in the sequence because they are just a list of notes: but Seq's sequences can involve randomness and external external MIDI control which changes what notes are output, so it can't do that.

MIDI Menu The MIDI Menu is where you set up MIDI to play and record. First, you with **Set MIDI Devices**, you can set up to 16 **Outputs** and up to four **Inputs** in your patch. Each Input or Output is a combination of a MIDI Device and a Channel. Once you have set up an input or output, it appears as an option in device selectors for Motifs which input or output MIDI. Inputs and Outputs also have **Nicknames**. If you give an Input or Output a Nickname, the Nickname will appear in the selectors for various Features instead of the device name. You get rid a Nickname by deleting the text in its text box.

When you select *Set* to save your choices, the Inputs and Outputs are saved not only with your patch, but also to global preferences so you next time you create a patch they'll be set by default. You can optionally select *Reload*, which reloads the Inputs and Outputs from your patch, clearing the changes you just made. If you load a patch, its current Inputs and Outputs will be loaded, overriding your defaults.

You can also **Panic**. This sends an **All Sounds Off** and an **All Notes Off** to every setup device in order to quiet MIDI devices with stuck notes.

Finally, you can **Log MIDI** to a file or generate a collection of **Notes** objects from it. If you start logging, then the next time you press Play, Seq will start logging a copy of all of its MIDI output. The logging stops when you press Stop. At that point, the logged MIDI is emitted to a single MIDI file, or broken out per-device to separate MIDI files,² or to a collection of Notes motifs, one for each device used. If more than one Notes is created, they will all be grouped under a single **Parallel** motif: if you play that Parallel as root, you'll hear the same music again. For more on Notes and Parallel, see Sections 2.1 and 2.5 respectively.

1.3 The Current Motif Display (and the Options and Per-Motif Menus)

The current motif display varies depending on which motif you have selected in the Motif List. Different motifs have different displays according to the nature of their data. We'll discuss different motifs and their displays later in Section 2.

Below the Current Motif Display are the **Motif Display Editing Buttons**. These vary depending on motifs and their need, though you might commonly see a button for adding elements, deleting them, or copying them. For example, in Figure 1, the Step Sequencer display has a "+" for adding a Track, a "-" for deleting the current track, and a button for duplicating the current track.

Also below the Current Motif Display are the **Display Zoom** buttons. These zoom the display in and out and again are only appropriate for certain motifs (notably the Step Sequencer motif).

The Per-Motif Menus Some motifs have their own dedicated menu which appears only when that motif is selected. These menus will be discussed with the motifs proper in Section 2.

The Options Menu This contains miscellaneous sequencer options. **Show Tooltips** toggles whether you want tooltips popping up everywhere. This option is remembered after you quit and restart Seq. **New Random Number Generator** gives Seq a random number generator with a new random sequence. And if **New Generator Each Play** is checked, Seq produces a new random number generator each time you press the Play or Record buttons: this is also remembered after you restart Seq. These last two menu items require some explanation.

Each sequence has a random number generator, which produces a stream of random numbers used whenever Seq has to roll the dice for something. Each time you make a new sequence, it is given its own unique random number generator, which is saved with the sequence. When you stop and replay the sequence, this generator is reset so you'll always have the same random choices made and thus will have the same sequence.

Normally this is desirable. But what if you don't like the stream of numbers you were given, and want a new random generator? Select "New Random Number Generator", and you'll get a new one. You can go back to the old one with **Undo**. If you save your sequence, it'll be saved with to use the new generator.

²When emitted to a single file, each device is written out as a separate channel. Some DAWs, such as Ableton, cannot read multi-channel MIDI files; hence the separate MIDI file option.

Note that if you change your sequence such that elements that draw random numbers are added, removed, or moved, you'll probably change the effect of the sequence on all the later random elements anyway. So don't expect the random series to be the same after you've made significant changes to your sequence.

1.4 The Inspectors and Text Notes

Each motif, and each element contained inside a motif, has an **Inspector**. An Inspector is a labelled list of every editable feature of the motif or element. Inspectors appear in the column at the far right of the window. Inspectors change a the Current Motif, or elements selected within the Current Motif, change.

At top right is the **Current Motif Inspector**. This is, unsurprisingly, the Inspector for the editable features of the Current Motif. Different motifs will have different available features, though they will all have at least *one* feature: the Motif's **Name**. By default the name is something boring like "Automaton 1", but you can rename it to anything you want.

Some Motifs also have **Parameters** their child motifs can refer to. Each of these parameters can be custom named, and the parameter names appear as an expandable region in the Current Motif inspector. We'll discuss parameters later in Section 4.

Below the Current Motif Inspector may appear **Element Inspectors**. For example, in Figure 1, below the Step Sequence inspector appear Inspectors for the currently selected Track in the Sequence, and for the currently selected Step in the Track. Many Motifs have Element Inspectors, but they vary. Motifs that hold Child Motifs typically show an Inspector for the currently selected Child, for example.

At bottom are **Text Notes**. You can resize this region and store all the notes you like, and they will be saved with each Motif.

2 The Motifs

Excluding Macros and Macro Children, there are presently nine Motifs available in Seq:

- Three **Basic (Note-Generating) Motifs**: Step Sequence, Notes, and Silence. These do not have children Motifs.
- Four **Combination Motifs**: Series, Parallel, Automaton, and Select. These have children Motifs.
- One **Parameter Modification Motif**: Modulation. This has a single child Motif.
- Two **MIDI Effect Motifs**: Arpeggio and Filter. These have a single child Motif each.

We'll be adding more Motifs in time.

Motifs are played through once, and when they have finished playing they do not themselves loop, but rather signal to their parent that they are **finished**. The parent then decides what to do with them, such as play them again, or go on to play something else, or signal to *its parent* that it is finished, and so on.

You can loop the entire sequence — that is, loop the **Root Motif** — by toggling the Loop button in the Transport prior to or during playback.

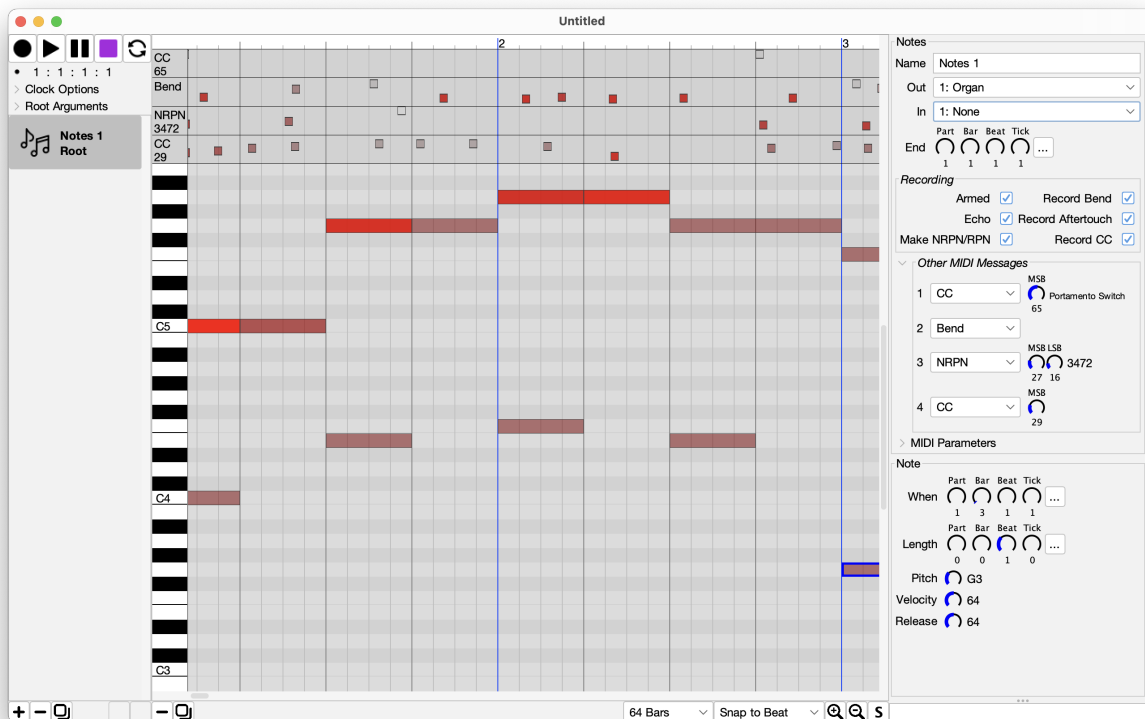


Figure 5 Notes Motif.

2.1 Notes

Notes is simply a collection of MIDI events: MIDI Notes, Pitch Bend events, Control Change (CC) events, Program Change (CC) events NRPN/RPN events, and Aftertouch Events. Notes are not stored as MIDI Note On/Note Off, but as entire notes with an onset, pitch, velocity, length, and release velocity. Other events are simply stored as their time stamp and value.

Like other Motifs, you can give your Notes a **Name** in the Notes Inspector.

The Note and Event Displays Notes are displayed using a **piano player roll** with scroll bars: the Y axis is pitch (with a little keyboard display at left to help you out) and the X axis is time. The color of the note is its **velocity**.

Other events are **optionally displayed** above the piano player roll. In the image above, you can see the time-stream of four such events. You select up to four event types to display with the **Non-Note Display** disclosure panel in the Notes Inspector at right. You can change the height of the event displays with **Parameter Height**.

The color of a non-note event is (at present) its **value**. The value is also shown with the *vertical position* of the event. Pitch Bend is a special case: although it has a large range (-8192 to $+8191$), very often subtle difference near 0 are important but can't be seen in the display due to resolution. By default Seq displays Pitch Bend warped (using square root), so values near 0 take up lots of space and large values take up rather little space. If you don't like this, you can turn it off by unchecking **Warped Pitch Bend** in the Non-Note Display disclosure panel. (Polyphonic Aftertouch is also a special case: its color reflects its pitch, not value.)

You can change the **maximum length of time** of the piano player roll with a pop-up menu at the bottom of the window, anywhere from 64 to 65536 bars. Making the length longer doesn't consume more resources,

and it's not slower. But it will make horizontal scrolling more annoying. So you'd like to keep it reasonably short when possible.

You can **zoom** the piano player roll in and out with the magnifying glass buttons at the bottom of the window.

At the top of the display is the **Ruler**. Depending on the zoom amount, will display ticks for Bars, Beats, and 16th notes. It will also show numbers for Bars.

Creating Notes and Events The create a note or an event, just double click in its region where you want it to be. The velocity and release velocity of the note will be set to the **Initial Velocity** and **Initial Release Velocity** respectively, found under the **Note Editing** disclosure panel in the inspector.

The position of the note or event, and the size of the note, are governed by the **snap** value, which you can set via a pop-up menu at the bottom of the window:

- **No Snap** The note/event onset will be placed right where you clicked. The note will be a quarter note (a beat).
- **Snap to/by 64th** The note/event onset will be placed at the nearest 64th note. The note will be a 64th note.
- **Snap to/by 16th** The note/event onset will be placed at the nearest 16th note. The note will be a 16th note.
- **Snap to/by Triplet** The note/event onset will be placed at the nearest triplet. The note will be a triplet.
- **Snap to/by Beat** The note/event onset will be placed at the nearest quarter note (beat). The note will be a quarter note.

Selecting and Adjusting Notes You can select a note by clicking on it. It will get a highlighted selection border in blue. Once you have selected a note, you can drag it to change its pitch or time. You can also change the length of the note by clicking and dragging the note near its right end.

You can also select multiple notes by clicking on the display and dragging a rubber-band around them.

If you hold down the **shift key** you can add and remove notes to your selection by clicking them or rubber-band dragging around them. You can do the same by **right clicking**.

If you hold down the **option** or **alt** key while dragging, or drag with the **middle mouse button**, then the selected notes will change in pitch but not in time.

If you have selected only one note, its Inspector will appear, and you can change its timestamp, length, pitch, velocity, and release velocity manually.

When you change the time or length of a note and you have **Snap To** turned on (see the pop-up menu at the bottom of the screen), it will **snap** to the nearest 64th note, 16th note, triplet, or beat. If instead you have **Snap By** turned on, it will **jump** forward or backward by one 64th note, 16th note, triplet, or beat in distance.

If you have lost your notes, just play the **Find Notes** button (the **"F"** at the bottom of the window, and the piano player roll will scroll to show your selected notes, or any notes at all (if you have nothing selected).

Selected notes can also be moved by through the **Up, Down, Left, and Right** menus and their corresponding keystrokes.

Selecting and Adjusting Non-Note Events Just like notes, you can select events by clicking on them, rubber-band selecting them (the rubber band will be a 1-dimensional range, not two-dimensional), and using the **shift key** or **right-click** in the same way. And if you have selected only one event, its Inspector will appear allowing you to change it more precisely.

You probably don't want to change both the time and value of a non-note event, so there are some constraints to help you out. When you drag a non-note event what happens depends on what keys you are holding down:

- **[Nothing]** Adjusts value but not time.
- **Option/Alt or Middle-Click** Adjusts time but not value.
- **Command/Windows** Adjusts both time and value.
- **Control** If adjusting value, all selected events are set to the same value.

Adjusting non-note Events in time follow the same **Snap To** and **Snap By** rules as for Notes. And the **Show Notes** facility also works for selected events.

Selected Events can also be moved by through the **Up, Down, Left, and Right** menus and their corresponding keystrokes.

Deleting and Duplicating You can delete your selection with the **Delete Button** (the minus sign) at the bottom of the window. You can duplicate your selection with the **Duplicate Button** next to it. You can also delete from an option in the **Notes Menu**

Playing Notes (and other Events) When Notes is played, it plays through and emits all its events in order exactly once, then stops. You can optionally set a **Start** point and an **End** point in the Inspector. Play will commence at the Start point, and will end after the End point, or after the last note or event in the sequence, whichever is later. Start can obviously be equal to or after End: in this case, play commences at start and ends as soon as the last note or event is over.

All notes and events are emitted out the same MIDI output specified by the **Out** in the Notes Inspector.

Recording Notes (and other Events) You can record notes from MIDI into the Notes. To do this you must first **Arm** the Notes: check the **Armed** checkbox in the Notes Inspector. Then start recording by pressing the record button. Recording will only start once it's the Notes's time to play. Notes will record MIDI from the MIDI Input specified by the **In** in the Notes Inspector.

If you check the **Echo** checkbox in the Notes Inspector, Notes will play to its Output all the notes it receives at its Input while recording. Note that whichever Notes motif was most recently open will control the Echo settings.

In the **Recording** disclosure panel there are a number of recording options. First, of you can specify the **Integration** of the notes and events after they are recorded:

- **Replace** means to delete all existing events and replace them with the newly recorded ones.
- **Replace/Trim** is the same, except that time is trimmed from the start so that the first recorded note or event starts at the first timestep. This is probably best if you also quantized.
- **Merge** means to simply add the new events to the existing ones (mixing them in).

Next, if you check the **Record Bend**, **Record Aftertouch**, or **Record CC** checkboxes, Notes will record these events: otherwise it will ignore and discard them while recording. And if you check the **Make NRPN/RPN** checkbox, Notes will attempt to build NRPN or RPN parameters as it receives them. NRPN and RPN parameters are made up of collections of CC messages, so if those CC messages are used to build a parameter, the CC messages will not be recorded as CC messages but as NRPN or RPN.

Other recording options deal with automatic quantization. You can turn **Quantize** on for notes, instruct it to also **Quantize Note Ends** and to **Quantize Non-Note Events** (like pitch bend or CC), and tell it to **Quantize To** a certain resolution. There also is a mysterious **Quantize Bias** knob. Normally Quantize works by rounding your note to the nearest resolution point, before or after. By adjusting the Quantize Bias knob you can push Quantize to more likely round to the note before or to the note after. This is useful because as humans we each have a different tendency to either play too early or too late, and can set the Quantize Bias knob to adjust for our own nature.

Recorded notes appear while recording. However recorded non-note events will not appear until after recording is finished, simply because there can be too many of them to draw efficiently.

The Notes Menu Notes has a number of items under its menu. Many of these items are applied to either all events or to a subset of them, depending on how you have selected notes or non-note events:

- **Selecting Notes or Events** will cause the menu item to only apply to the selection.
- **Setting a Range in the Ruler.** You can drag in the ruler to create a range of time. Menu items will apply to *all events*, both note and non-note events, within that range. You can deactivate the ruler range by clicking in the ruler. Note that the ruler range does not effect Deletion or Copying: they require selection.
- **All Events and Notes.** If nothing is selected, and there is no ruler range set, then the menu item will apply to all notes.

The menu items are:

- **Select All Notes** selects all notes.
- **Cut** and **Copy** do what you expect. They cut/copy either the selected events, or all events which fall inside the Ruler range.
- **Paste** also does what you expect: it pastes the cut/copied notes, merging them into the existing notes. But where do the Pasted events go? If you have selected a range in the Ruler, Paste will paste its events starting at the beginning of that range. If you have selected notes or events, Paste will paste its events immediately after the selection. If no notes or events are selected, and the ruler has no range, Paste will paste its events right where they were cut or copied. This is intentional, though it might confusing as they'd be on top of their previous notes/events.
Note that you can cut/copy from one Notes motif and Paste into a different one.
- **Paste/Replace** is just like Paste, except that the selected events, or those in the Ruler range, will be deleted and replaced with the pasted events.
- **Replicate** copies the selected or range events and places that copy in its own new Notes motif. The events are trimmed so that they start at timestep 1..
- **Quantize...** gives options for quantizing all or a selection/ruler range of your notes or event data.
A note about the **quantization bias**. What is this? Normally quantizers just round the notes up or down to the nearest quantized position, but this is often wrong. As a human, you may tend to play more notes early than late (or vice versa), resulting in an error distribution which is *ought* to favor the earlier quantized positions more than the later one (or vice versa). You can set the bias to match your personal tendencies.
- **Stretch Time...** stretches the time of all of your data or the selection/ruler range.
- **Trim Blank Time from Start** removes empty time from the start of your data. Your first event will start at timestep 1 in the Notes.
- **Randomize Time...** gives options for adding noise to the onset and optionally the duration of all of your data or the selection/ruler range.
- **Set Velocity...** lets you set the velocity of all notes or those in the selection/ruler range.
- **Randomize Velocity...** gives options for adding noise to the velocity of all notes or those in the selection/ruler range.
- **Interpolate...** adds additional non-note events in-between the selected events to gradually shift the event value from one to the other. You can interpolate among more than just two events. You can't interpolate notes.

- **Delete Selection** deletes the selected events. Note that this does *not* delete events within the ruler range.
- **Filter...** gives options for filtering out (deleting) notes, bend data, CC data, RPN and NRPN data, or aftertouch data. Since selection is applied to a single kind of data, Filter is really is only appropriate for filtering all data, or data within the ruler range (not selection).
- **Move to Top** sets the selected notes so that they display on top of other notes.
- **Send to Back** sets the selected notes so that they display behind other notes.
- **Arm New Notes Motifs** sets it so that new Notes Motifs will automatically be armed (or not).

Parameterization Several Notes features (note velocities, release velocities, event values) can be **Parameterized** by double-clicking on the dials. If a note velocity or event value is parameterized, the note or event is displayed in gray-blue to indicate this. For more on parameterization, see Section 4, (*Parameterized Control of MIDI Data*).



Figure 6 Step Sequence Motif.

2.2 Step Sequence

Step Sequence is one of the most complex motifs, as it is a reasonably full-featured multitrack step sequencer. Step Sequence defines a single multitrack pattern, which is played once and then declares it is finished. The Step Sequence can have any number of Tracks, and each Track can have any number of Steps (and that number can vary from Track to Track). By default the Step Sequence presents itself with 16 Tracks of 16 Steps.

You select a Step by Command- or Alt- (or Option-)clicking on it. This also selects the Step's Track. You can also select a different Track (and its equivalent Step) by clicking on the Track Header (see below). When you select a Step and Track, they will show their respective Inspectors at right below the primary Step Sequence Inspector.

You can toggle a Step on or off, by clicking on the step. This also selects it. Thus another way to select a Step (but not toggle it) is to double-click on the Step, which effectively toggles it twice and selects it.

If you triple-click on a Step, you will clear all the Steps in its Track.

Defaults and Parameterization Several of the dials the Step Sequence's Tracks and Steps can be set to a fixed value or to **Default**. In a Step, Default means that instead of being fixed to a value, the dial defers to the Track's setting. In a Track, Default means that instead of being fixed to a value, the dial defers to the Step Sequence's overall setting.

To set a dial to Default, double-click on it, and select *Default* from the resulting pop-up menu. To set a Defaulted dial back to a fixed value, just change the dial. You'll note that the pop-up menu has other values, such as Rand or Param 1 ... Param 8. We'll discuss those in Section 4 (**Parameterization**). Don't set to those for now. Furthermore, some menus don't have a Default, or no pop-up menu even appears: this means that

the dial doesn't have a Default. You can tell that a dial has a pop-up menu because it has a red or gray **dot** in its center.

Steps A Step is either **off** (gray) or **on**. When on, the Step has a **velocity** (volume), represented by a graduated color between white and red.³ Each Step can also have its own **Note Pitch**, its own number of **Flams** (ratchets), and a declaration of **When** (or perhaps better called *whether*) the Step plays when its time is up. The When setting can be *always*, or it can play with some *probability*, or it can play with some *pattern*. An example pattern is O O X, which means to play the first two times the Step Sequence is iterated, and then not play the third time, and then repeat.

All four of these features (Note, Velocity, Flams, and When) also can be set to the special value **Default**.

Every fourth Step is bolded in the interface only for visual assistance. The currently selected Step is outlined in blue.

Tracks A track is an iteration of Steps played one after another. Even though tracks can have different numbers of steps, they will play their steps such that they all finish at the same time.

A Track has a **Track Header** and then the list of Steps. You can change the order of a Track in the list of Tracks by dragging its Track Header up or down. A Track also has a **Name**, which you can set in its Inspector. Track Headers have three buttons: **Learn (L)**, **Solo (S)**, and **Mute (M)**. If you select Mute, then that track will not play. If you select Solo, then *only* that track will play. If you select Learn, then the track is **learning** its track note while playing. This means that if you press a key, the track will use that key as its output **Note**.

You can add, delete, and duplicate tracks with the buttons at bottom. You can also Zoom In and Zoom Out. The currently selected Track has its track number highlighted in red.

Tracks have a number of features in their Inspectors, beyond their **Name**.

All the Steps in a given Track play their MIDI out the same MIDI Out as specified by the Track. By default this is set to the special value *Default*, meaning that the Track defers to the Sequence's global setting. A track also has a **Gain** which is multiplied against the velocities of its Steps to determine their final volume.

A track also has a default **Note**, **Velocity**, number of **Flams** and **When** for its Steps. By default Steps defer to these values, but you can of course change them on a per-Step basis. You can also press **Set All**, which causes all Steps to reset themselves to Default for that feature.

Each track also has up to one **Choke Track**: when a Step in the Track is played, any playing sound in the Choke Track is immediately stopped (a MIDI Note Off is issued). Tracks also have some degree of **Swing** (syncopation). A typical amount of swing would be about 30% (if you wanted Swing), else 0%.

A Track can be a member of the **Exclusive Random Group** (or "Ex. Rand."). When the Sequence is played, one and only one Track in the Exclusive Random Group is played; the others are hushed (independent of Mute settings). The Track in question is selected at random. Tracks not in this group are not effected.

Finally, a track has a **Length**, that is, the number of steps. This is not set until the **Set** button is pressed.

Euclidean Rhythms In 2004 Godfried Toussaint published a paper suggesting that large numbers of common rhythms worldwide could be automatically generated using Bjorklund's Algorithm, which is somewhat related to Euclid's Algorithm. He called these **Euclidean Rhythms**. Bjorklund's Algorithm takes a track K steps long and attempts to fill it with N steps that are statistically evenly spaced from one another. So there are two parameters, $0 < N < K$. Euclidean rhythms have since become all the rage.

Seq can fill a track with a Euclidean Rhythm with **Euclid**. You specify N , the number of steps you want on (the rest will be off. K is just the length of the track. You can also **Rotate** your Euclidean Rhythm to a new starting point. The Rhythm is generated when you press **S** (for Set). You can also generate a totally random Rhythm by pressing **R** (for Random).

³Note that because MIDI interprets velocity=0 as a Note Off message, which we do not want, Seq will treat a velocity of 0 as a (very low) velocity of 1.

The Step Sequence Inspector At top, the Step Sequence inspector defines the global settings for the Step Sequence. There is its **Name** of course. After this there is the **Duration**, that is, the length of time (in 16th notes) that this Step Sequence will take to complete. For example, even if the Step Sequence is 16 steps long, if you set the Duration to 4, it'll run at the speed necessary to complete those 16 steps in one quarter note of time. The default is of course 16: 16 steps in 4 quarter notes.

You can also set the default **Swing**, **Velocity**, and **Out**, and Tracks will defer to these if set to *Default*. The **Track Length** indicates the initial number of Steps for newly added Tracks. Finally, **In** specifies the MIDI Input to use when recording tracks.

Sequencing Things Other than Notes The Step Sequence inspector also has an option called **Type**. By default, this is set to "Note", but you can set it to other kinds of MIDI types, such as CC or NRPN. This affects the meaning of the "Note" and "Velocity" dials, and also each step's Velocity coloring, as follows:

- **Note** Nothing changes.
- **CC (Control Change)** "Param" represents the CC parameter. "Value" represents the CC value.
- **Poly Aftertouch** "Param" represents the polyphonic aftertouch note. "Value" represents the aftertouch value.
- **Channel Aftertouch** "Param" does nothing. "Value" represents the CC value.
- **Pitch Bend** "Param" does nothing. "Value" represents the MSB of the pitch bend value, and its "LSB" represents the LSB of the value. The final pitch bend value, which ranges from -8192 to +8191, is defined as $MSB \times 128 + LSB - 8192$. We recognize that Pitch Bend is defined in an inconvenient way. We're working on it.
- **PC** "Param" does nothing. "Value" represents the PC value.
- **NRPN** "Param" represents the MSB of the NRPN parameter, and its "LSB" represents the LSB of the parameter. The final NRPN parameter, which ranges from 0 to 16383, is defined as $MSB \times 128 + LSB$. "Value" represents the MSB of the NRPN value, and its "LSB" represents the LSB of the value. The final NRPN value, which ranges from 0 to 16383, is defined as $MSB \times 128 + LSB$.
- **RPN** Similar to NRPN.

The Menu The **Step Sequence** menu allows you to use keystrokes make your way around the sequence (**Up**, **Down**, **Left**, **Right**). You can also **Toggle** the selected step to be on or off. And you can **Rotate** all the steps in the selected track **Left** or **Right** by one step.

Under the **Randomize** submenu are six options for randomizing the pattern of the current selected track or of all tracks. They're set up so you can easily select them with keystrokes.

Parameterization Several Step Sequence features (step velocities and pitches, swing, etc.) can be **Parameterized** by double-clicking on the dials. If a step's velocity is parameterized, the step is displayed in gray-blue to indicate this. For more on parameterization, see Section 4, (*Parameterized Control of MIDI Data*).

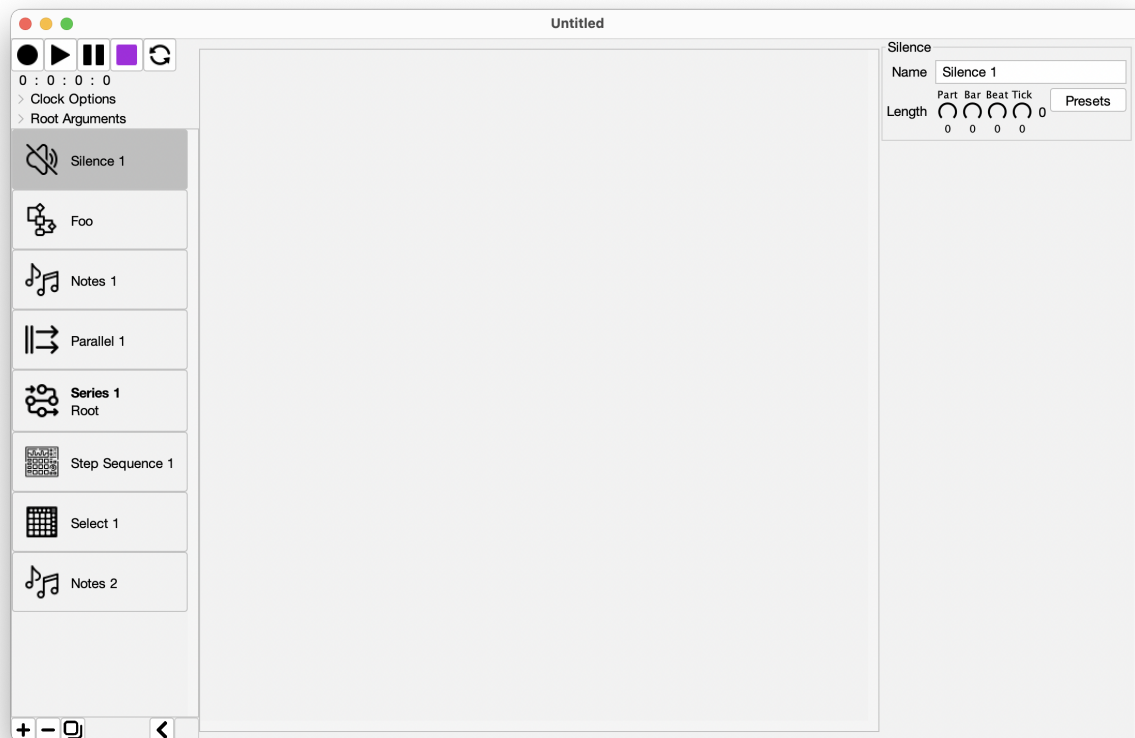


Figure 7 Silence Motif.

2.3 Silence

Silence is nothing more than a fixed-length chunk of silence. You can specify the **length** in parts, bars (measures), beats, and ticks. There are 192 ticks per beat, and 256 bars per part. The number of beats per bar depends on your clock settings.

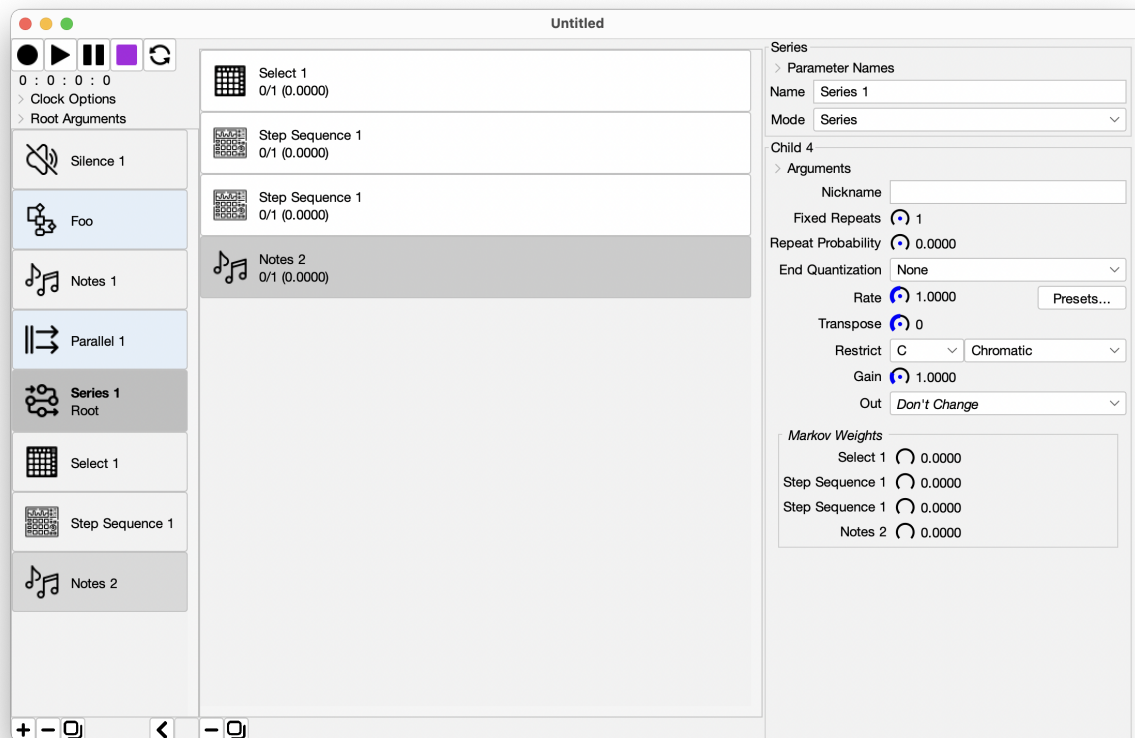


Figure 8 Series Motif.

2.4 Series

By default Series is exactly what it suggests: a series of child motifs played one after another. By “Play” we mean that a Child Motif is played until it is finished, and then this is repeated some N times. That is, Series is essentially Song Mode.

But in fact Series has several modes, and so can be set to play its children in a number of ways:

- **Series** Play each child in sequence, then finish.
- **Shuffle** Randomly shuffle the order of the children. Then play them in that order, then finish.
- **Random Forever** Select a random child to play. When finished playing it, select another one at random and play it, and so on. Do this forever.
- **Random Markov** Select a random child to play. When finished playing it, select a second one at random using the first child’s Markov Weights, and play it. Then select a third child at random using the second child’s Markov Weights, and so on. Do this forever.
- **Round Robin** Play the first child. When finished playing it, finish. If you are immediately looped back to playing again, play the second child, then finish, and so on, eventually returning back to playing the first child. Note that if you’re not immediately being looped, you won’t increment to the next children, but will just play the first child over and over.
- **Variation** Play the child specified by Parameter 1, then finish. (For discussion of Parameters, see Section 4). This is particularly useful for creating and playing multiple *variations* of a given Motif.

- **Random Variation** Play the child specified by the Random Parameter, then finish. (For discussion of Parameters, see Section 4).

The **Mode** is specified in the Series Inspector.

Children In the Current Motif Display, Series maintains a list of Child Motifs that it plays in some order or otherwise selects from. You can drag Motifs into this list directly from the Motif List. You can drag the same Motif to appear multiple times in the Child list. You can also rearrange the Motifs in this list by dragging them, and can delete or duplicate a Motif in the list.

While a Child is playing, its name is highlighted in red in the Child List.

Child Features Each Child in the Child Motifs can have a **Nickname**, set in its Child Inspector. Even if a Motif appears multiple times in the List, each entry can have a different Nickname. When you set the Nickname, it appears with the Child in the list.

When a Child finishes playing once, we can immediately go on to whatever is next to do, or we can first wait until some quantized time boundary (16th note, quarter note, or measure) before doing so. This is set in the Inspector as **End Quantization**.

When a Child is played, it is played once, and then for some number of additional iterations or **Initial Repeats**. You can set this value in the Inspector. Thereafter, we flip a coin of **Repeat Probability**, and each time it comes up heads, we play the Child once more. When it comes up tails, we stop playing the child, and go on to the next step (starting playing some other child, or finishing). These two values are also displayed in the Child in the list, along with the number of iterations performed so far.

Alternatively if you select **Until Trigger 8**, both Repeats features are ignored and the Child will repeat forever until the Series receives a Trigger on Parameter 8. (For discussion of Parameters, see Section 4).

One Child is designated the **Start**. You choose it by selecting “Start Here”. If you are in *Series Mode*, and the Series is the *Root*, then when you press play, playing will begin with that Child. If no Child is designated the Start, then the Start is 0. The purpose of this mechanism is to temporarily allow you to change where in the song you’d like to start playing or recording.

MIDI Modification As a Child of the Series emits MIDI, that MIDI is handed to the Series, which has a chance to modify it before it is passed on. There are several available features for modification of a given Child’s MIDI in its Inspector. **Transpose** transposes all notes by some number of semitones up or down. **Restrict** forces the notes to line up with notes in a given scale: notes that don’t match are transposed down to the nearest note in the scale. **Gain** changes the overall volume (velocity) of notes. And **Out** changes the MIDI Out being used (by default it is set to *Don’t Change*).

Finally, **Rate** changes the speed at which the child is played, faster or slower. Keep in mind that Seq has a resolution of 192 ticks per beat (PPQ), so changing the speed — particularly faster — can quickly result in inaccuracies. Some **Presets** are provided, including a Custom Rate option.

Markov Weights If the Series’s mode is Random Markov, the first child to play is selected entirely at random, but each subsequent child is selected using the Markov Weights of the current child. The Markov Weights of a child are part of its features. For a given child, each subsequent child is assigned a weight from 0.0 to 1.0 indicating the probability that it will be selected next time. The weights are normalized to form probabilities.

For example, if you have three children, A, B, and C, and you have set A’s Markov Weights to be A: 0.5, B 0.5, C 0.0, then after A has finished playing, either A or B will be selected next with equal likelihood.

Parameters and Arguments The Series Inspector has an expandable region called **Parameter Names**, and each Child Inspector has an expandable region called **Arguments**. Furthermore, many dials have Special Values called “Rand”, “Param 1”, etc. These are explained in Section 4.

The Series Inspector also has an expandable region called **MIDI Parameters**. These are discussed later in Section 4.1 (*Parameterized Control of MIDI Data*).

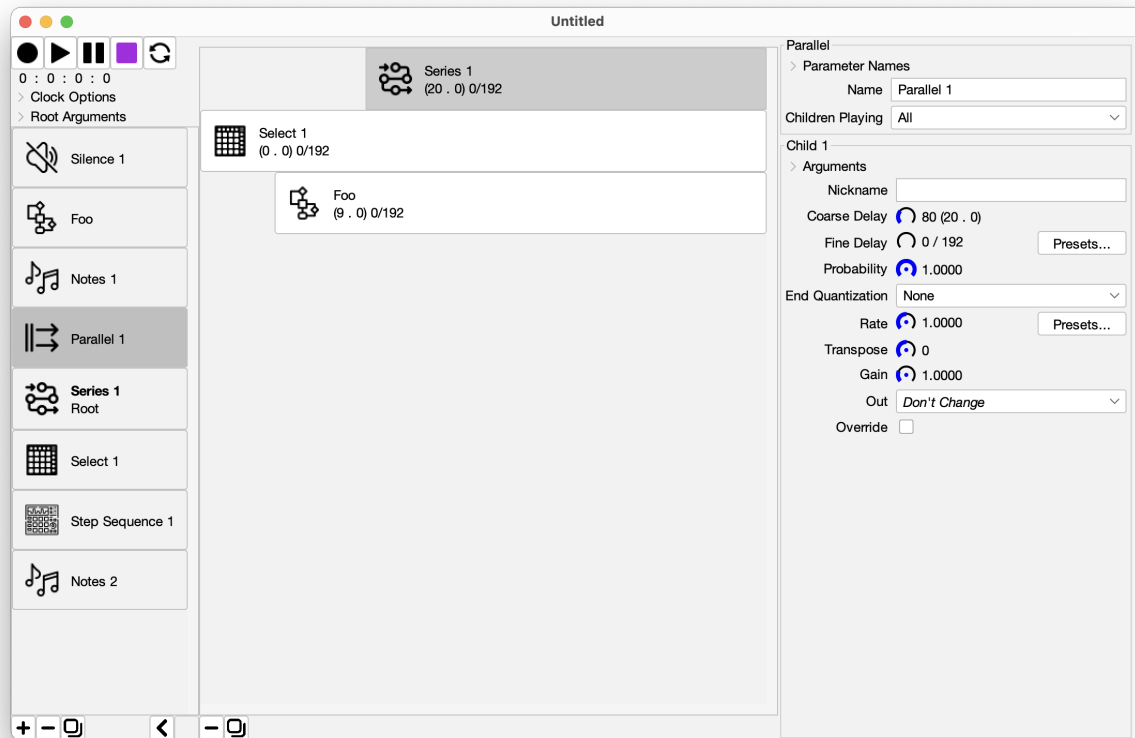


Figure 9 Parallel Motif.

2.5 Parallel

Like Series, Parallel is a list of Children; but it plays them in parallel. Each child can have its onset delayed, so that they start at different times. Each child can have some probability as to whether it is played at all.

Children In the Current Motif Display, Parallel maintains a list of Child Motifs that it plays in some order or otherwise selects from. You can drag Motifs into this list directly from the Motif List. You can drag the same Motif to appear multiple times in the Child list. You can also rearrange the Motifs in this list by dragging them, and can delete or duplicate a Motif in the list.

While a Child is playing, its name is highlighted in red in the Child List.

If a Child is playing repeated (looping) its name is highlighted in blue in the Child List.

Each Child in the Child Motifs can have a **Nickname**, set in its Child Inspector. Even if a Motif appears multiple times in the List, each entry can have a different Nickname. When you set the Nickname, it appears with the Child in the list.

Cross-Fading Parallel can be used as a real-time cross-fader between the first two children. The value of **Cross-Fade** determines the gain of Child 1 versus Child 2. If Cross-Fade is 0.0, then Child 1 plays at full gain and Child 2 is silenced. If Cross-Fade is 1.0, then it's the opposite. The default is 0.5, where each child plays at half gain.

By fault Cross-Fading is disabled, regardless of what **Cross-Fade** says. To enable cross-fading, check the **Cross-Fade On** checkbox.

You can further adjust the gain of each of the children with the **Gain** option in each child's inspector. This gain is multiplied against the cross-fade gain result (if the cross-fade gain is enabled).

Play Probability Each Child has a **Probability** in its Inspector. This probability determines whether the child will be played as follows. In the Parallel Inspector there is a feature called **Children Playing**. This feature allows you to select some N children to play when the Parallel starts playing. Its options are:

- **All** Each child will play with its given its Probability (by default 1.0). This is the default.
- **1...16** Parallel will select some N (1...16) children at random given their Probabilities. These will be the only children that play.
- **All, Finish After First** Every child will play with its given its Probability, except the first child in the list, which always plays. After the first child in the list has finished playing, all children will immediately be terminated.

If a child has **Override** set in its Inspector, then when it starts playing, it will immediately terminate all Children below it.

Maximum End Time You can set the maximum end time of the Parallel with **Max End Time**. This cuts off all Children if they exceed this time. Children (and the whole Parallel) can end sooner than the Max End Time, however.

Delays and Quantization Each Child can have the onset of its playing delayed. Seq provides a **Start** time in Parts, Bars, Beats, and Ticks, though you'll probably usually use Beats and Bars.

When the last Child finishes playing once, we can finish Parallel immediately, or we can first wait until some quantized time boundary (16th note, quarter note, or measure) before doing so. This is set in its Inspector as **End Quantization**.

When a Child is delayed, it will shift horizontally to demonstrate this, as is shown in Figure 9.

The Ruler At the top of the display is the **Ruler**, which shows measures and, if zoomed in, beats and sixteenth notes. You can zoom the Parallel in and out with the magnifying glass buttons at the bottom of the window.

Repeating After a Child has played, you can set it to **Repeat**, that is, to play in a loop until all of the children have been given a chance to play at least once: basically it's vamping. The Parallel will terminate after all Children have terminated or are looping.

While a Child is repeating, its button text color will change to blue and it will say "REPEATING".

Why is this useful? Let's say you want a Step Sequence to repeat its (shorter) pattern as long as a Notes melody is playing. That's a very common need. There are various ways to do this, but the easiest way is to put them both in a Parallel, and set the Step Sequence to be Repeating.

MIDI Modification As a Child of the Parallel emits MIDI, that MIDI is handed to the Parallel, which has a chance to modify it before it is passed on. There are several available features for modification of a given Child's MIDI in its Inspector. **Transpose** transposes all notes by some number of semitones up or down. **Restrict** forces the notes to line up with notes in a given scale: notes that don't match are transposed down to the nearest note in the scale. **Gain** changes the overall volume (velocity) of notes. And **Out** changes the MIDI Out being used (by default it is set to *Don't Change*).

Finally, **Rate** changes the speed at which the child is played, faster or slower. Keep in mind that Seq has a resolution of 192 ticks per beat (PPQ), so changing the speed — particularly faster — can quickly result in inaccuracies. Some **Preset**s are provided, including a Custom Rate option.

Unlike in other Motifs, you can **Mute** children in a Parallel. The children will still have the same length and override: they just won't produce any notes. They will produce CC and similar data (for now).

Parameters and Arguments The Parallel Inspector has an expandable region called **Parameter Names**, and each Child Inspector has an expandable region called **Arguments**. Furthermore, many dials have Special Values called "Rand", "Param 1", etc. These are explained in Section 4.

One trick you can do with Probabilities: bind certain child probabilities to an argument. Then in a parent you can mute the child at certain iterations by setting that argument to 0.0 versus 1.0.

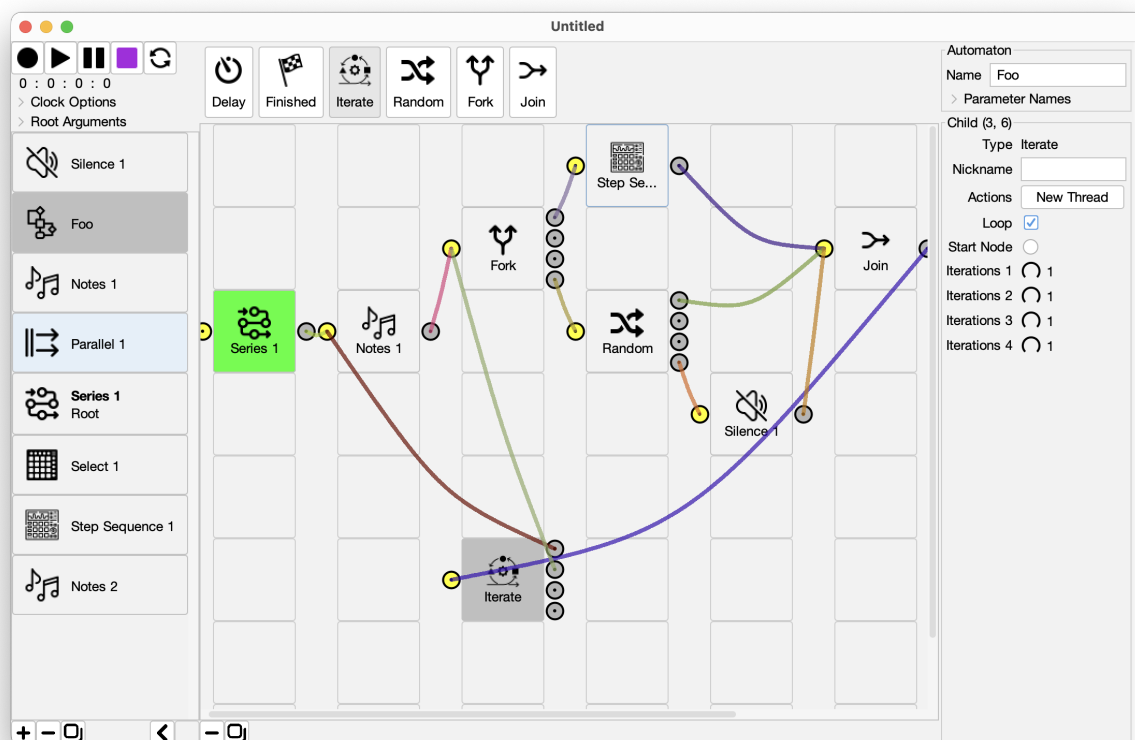


Figure 10 Automaton Motif.

2.6 Automaton

Automaton may be thought of as an elaborate combination of Parallel and Select. In Automaton you lay out Children, and various decision point nodes, then connect them to construct a diagram indicating the process you want to take place. This is known as an *Automaton*.

An Automaton contains **Nodes**, that is, Children and Decision Points — and **Edges**, that is, connections from Node to another. Connections go from the endpoint of a Node to the start point of a new Node. In our Automaton, several kinds of Nodes can have multiple endpoints, but all Nodes have just one start point. However, while can connect many Edges to the same start point, you can connect only one Edge to a given end point.

You add Children to the Automaton by dragging them in from the Motif List. You add Decision Points to the Automaton by dragging them in from one of the six Decision Point buttons at top (**Delay**, **Chord**, **Finished**, **Iterate**, **Random**, **Fork**, and **Join**). You can delete a Node by selecting it, then pressing the **Delete Button** at bottom, and similarly you can duplicate a Node by selecting it, then pressing the **Duplicate Button**. You can move nodes to different slots by dragging them.

Edge are connected by dragging from an endpoint to a start point. You can delete an edge by clicking on the endpoint from which it began.

Playing The Automaton is capable of playing multiple things at one time. The process of playing a single item is called a **Thread**. In a Thread, a single Node is playing, and when it finished playing, it passes playing on to another Node, or if it passes to nothing, the Thread is terminated. Threads operate independently. There can be no more than 8 Threads playing at one time. Normally Threads are created automatically, but

you can manually start a Thread on any Node by selecting the Node and selecting **Launch Thread at Node** in the **Automaton** menu (if there are already 8 active threads, selecting this menu item does nothing).

One Node is designated the **Start Node**. At present it's colored ugly Lime Green. When the Automaton is started, it initiates a single Thread, playing the Start Node. You can designate the Start Node by selecting it and then selecting **Make Start Node** in the **Automaton** menu. You can do the same thing by pressing the **Start Button** (the small button labelled "S" at the bottom right of the Motif Display).

When a Node has finished playing, it will select one of its endpoints, and its Thread will start playing whatever is connected to that endpoint via an Edge. If nothing is connected, then nothing new will play and the Thread will terminate. If all Threads have terminated, then the Automaton itself will Finish: there's nothing left to play!

Some nodes take up time while playing: for example, a Notes object will take time to play its notes. Indeed all Children must consume at least one Tick of time. However many Decision Point Nodes take up zero time to play, and immediately transfer to other Nodes. It's possible to set up a cycle (an infinite loop) of zero-time Decision Point Nodes, but Seq will detect this and force the to consume at least Tick of time.

Child Features Each Child has a **Nickname** that you can set in its Inspector to make it easier to read.

While playing, a Child is played once, then played additional iterations, as designated by its **Initial Repeats**. Thereafter we flip a coin with a **Repeat Probability** and each time it comes up heads, we play the Child again. When it comes up tails, we stop playing the Child.

Alternatively if you select **Until Trigger 8**, both Repeats features are ignored and the Child will repeat forever until the Series receives a Trigger on Parameter 8. (For discussion of Parameters, see Section 4).

Each iteration of the Child stops (or restarts) at the nearest **Quantization** after it has finished. This can be *None* (no stop immediately), *16th Note*, *Quarter Note*, or *Measure*.

The Decision Points Decision Points are not Children: when they are playing they do not cause other Motifs to play. Instead, they are used to control the flow of the Automaton process. Each Decision Point has a **Nickname** that you can set in its Inspector to make it easier to read.

There are six Decision Point Nodes at present:

- **Delay** While playing, this simply does nothing for a set period of time, then finishes. You can set the amount of **Delay** in Ticks, Beats, Bars (Measures), and Parts. There are 192 Ticks per Beat, and 256 Bars per Part. The number of Beats per Measure is set in the Clock Options. There are **Presets** in fractions of a Beat that you might find helpful. Delay and Chord are the only Decision Points that takes up time.
- **Chord** While playing, this plays a chord. You can set the total **length** in You can set the **Length** in Ticks, Beats, Bars (Measures), and Parts. There are 192 Ticks per Beat, and 256 Bars per Part. The number of Beats per Measure is set in the Clock Options. There are **Presets** in fractions of a Beat that you might find helpful. Delay and Chord are the only Decision Points that takes up time.

You can also set the **Gate Percentage**. This is the percentage of the Length where the Chord actually plays. You can specify both the **Velocity** and the **Release Velocity**, and of course up to four **pitches** for the chord notes proper.

- **Finished** This simply asks the Automaton to declare to its parent that it is Finished, and then terminates in zero time.
- **Iterate** This Decision Point is used to perform elaborate controlled loops. Iterate has four endpoints. When it starts playing, it terminates immediately in zero time and control continues out one the first four endpoints that is connected. When you start playing it *again*, it does the same thing. This continues for the number of **Iterations** designated for that endpoint. When these iterations have been completed, it will start outputting at the *next* endpoint that is connected, and so on.

If there are no more endpoints, and **Loop** has been selected, then Iterations will loop back and start again with the first connected endpoint. Otherwise it will terminate and not pass control any more.

Iterate normally updates its position in the loop regardless of which Thread is passing through the Iterate at the moment. However you can change this behavior by checking **Local**. Now each Thread will have its own separate Iterate position, rather than updating a common one. This may produce behaviors you didn't expect if you're not keeping track of which Thread is which, so you may not want to check this.

- **Random** This Decision Point is used to make random decisions. Random has four endpoints. When it starts playing, it terminates immediately in zero time and control continues out one of its four endpoints. The endpoint selected is done so at random using the **Weights** as probabilities. If the selected endpoint is connected, the connected Node starts playing. Otherwise, Random terminates and does not pass control.
- **Fork** This Decision Point passes control to *all its endpoints at the same time*, thus allowing playing of multiple Nodes in parallel. It does this in zero time. New Threads are created as necessary. If Fork cannot create enough Threads (the active thread limit of 8 would be exceeded) Threads are only created up to the maximum for the earlier endpoints, and later endpoints are ignored.
- **Join** This Decision Point waits until some number *N* of Threads have reached its start point, and then it terminates all of them, and creates a single Thread which passes control to its endpoint. You can set *N* to any value from 2 to 8 (2 is the default). Join is used to enable playing Threads to synchronize and wait for one another before terminating.

MIDI Modification As a (Motif node) Child of the Automaton emits MIDI, that MIDI is handed to the Automaton, which has a chance to modify it before it is passed on. There are several available features for modification of a given Child's MIDI in its Inspector. **Transpose** transposes all notes by some number of semitones up or down. **Restrict** forces the notes to line up with notes in a given scale: notes that don't match are transposed down to the nearest note in the scale. **Gain** changes the overall volume (velocity) of notes. And **Out** changes the MIDI Out being used (by default it is set to *Don't Change*).

Finally, **Rate** changes the speed at which the child is played, faster or slower. Keep in mind that Seq has a resolution of 192 ticks per beat (PPQ), so changing the speed — particularly faster — can quickly result in inaccuracies. Some **Preset**s are provided, including a Custom Rate option.

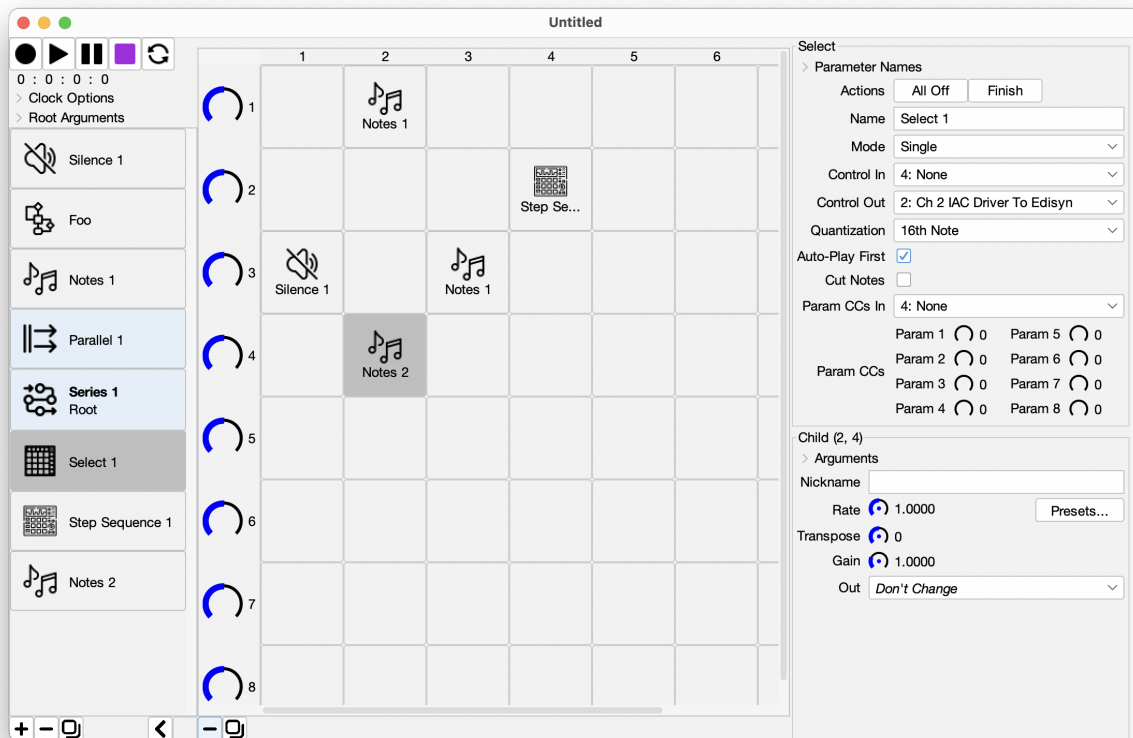


Figure 11 Select Motif.

2.7 Select

Select is meant to add performance facilities to Seq. Select is different from Series, Parallel, and Automaton: it has Children, but you as a user get to select which ones you want to play in real-time. This can be done either via the user interface, or by connecting a Novation Launchpad MK I or MK III, or an Akai MPK Mini. Seq also provides facilities for changing parameters, including large dials, connections to remote controllers via Control Change messages, and a large virtual joystick.

Select defines a grid where you can place Children by dragging them in from the Motif List. The location of the Children does not matter; you can lay them out however you like. However if **Auto-Play First** is selected in the Select inspector, then the first child (most top-left) will be automatically played when Select is first played.

You can drag a Child to a new location, and you can also delete it by selecting it and pressing the **Delete Button** at bottom, and similarly you can duplicate a Child by selecting it, then pressing the **Duplicate Button**.

When you click on a Child, it is selected as usual and its Inspector will come up. But it is not chosen to start playing. How do you do that then? By either middle- or right-clicking on it, or holding down any modifier key (Command, Control, Shift, Option, Alt) and clicking on it.

Each Child has a **Nickname** which you can set in its Inspector. This is useful for making a shorter name for it in the grid.

Select has four modes:

- **Single** Only one Child may play at a time, but only plays once, then terminates. If you choose a Child to play, the old Child will stop and the new one will start. If you select a Child which is already playing, it will play again when it has finished this time.

- **Single Repeating** Only one Child may play at a time, and does so over and over. If you choose a Child to play, the old Child will stop and the new one will start. If you select a Child which is already playing, it will stop playing.
- **Multi** Multiple Children may play at a time, but each only plays once, then terminates. If you choose a Child to play, it starts playing. If you select a Child which is already playing, it will play again when it has finished this time.
- **Multi Repeating** Multiple Children may play at a time, and each does so over and over. If you choose a Child to play, it starts playing. If you select a Child which is already playing, it will stop playing.

Quantization and Starts and Stops When a new Child is selected to play, and the **Quantization** is set to *16th Note*, *Quarter Note*, or *Measure*, then the new Child will wait until we reach the next time Quantization before it begins playing. In the meantime, it is colored Blue.

When a new Child is selected to play, another Child may need to stop first. If the **Quantization** is set to *16th Note*, *Quarter Note*, or *Measure*, then the old Child is allowed to finish playing first (and we wait until we reach the next time Quantization). However if **Quantization** is *None* then the old Child is terminated immediately.

Similarly, if we have terminated a Child, Quantization determines when it will stop.

Other Features Select has an action button, **Finish**. Pressing Finish will cause Select to tell its parent that it has Finished. This is a rare need.

MIDI Modification As a Child of the Select emits MIDI, that MIDI is handed to the Select, which has a chance to modify it before it is passed on. There are several available features for modification of a given Child's MIDI in its Inspector. **Transpose** transposes all notes by some number of semitones up or down. **Restrict** forces the notes to line up with notes in a given scale: notes that don't match are transposed down to the nearest note in the scale. **Gain** changes the overall volume (velocity) of notes. And **Out** changes the MIDI Out being used (by default it is set to *Don't Change*).

Finally, **Rate** changes the speed at which the child is played, faster or slower. Keep in mind that Seq has a resolution of 192 ticks per beat (PPQ), so changing the speed — particularly faster — can quickly result in inaccuracies. Some **Presets** are provided, including a Custom Rate option.

Parameters and Arguments The Select Inspector has an expandable region called **Parameter Names**, and each Child Inspector has an expandable region called **Arguments**. Furthermore, many dials have Special Values called "Rand", "Param 1", etc. These are explained in Section 4.

Overriding Parameters Select allows you to override parameters sent to certain children so you can control them by turning a big Dial or via a MIDI CC from a remote controller. This is discussed later in Section 4.2.

The Joystick You can assign two parameters to a Joystick available in the Inspector. The parameters are assigned using the "X" and "Y" radio buttons in the Parameters section of the Inspector (again, see Section 4.2).

Using a Grid Controller To use a Grid Controller to select Children, you specify it in the **Control In** and **Control Out**. Specify the device type in **Device**. When the Select starts playing, it will reset the Launch Pad to display the same grid as your current user interface layout. You can then choose Children by pressing buttons on the Grid Controller.

Available Grid Controller Devices

- **Novation LaunchPad MK I Series** LaunchPad, LaunchPad S, LaunchPad Pro. Only tested on the LaunchPad S.
- **Novation LaunchPad MK III Series** Launchpad Mini, Launchpad X, LaunchPad Pro [MK3]. Only tested on the LaunchPad Mini. When using a MK3 device, you should use the Novation LaunchPad's **MIDI IN** and **MIDI OUT** devices, not its **DAW IN** nor **DAW OUT** ones.
- **Akai APK Mini MK2** Mini MK2.

Troubleshooting

When I go to my Select, it plays a note on my synthesizer even when stopped.

You have the Control Out set to your synthesizer device. Instead, set it to your Grid Controller.

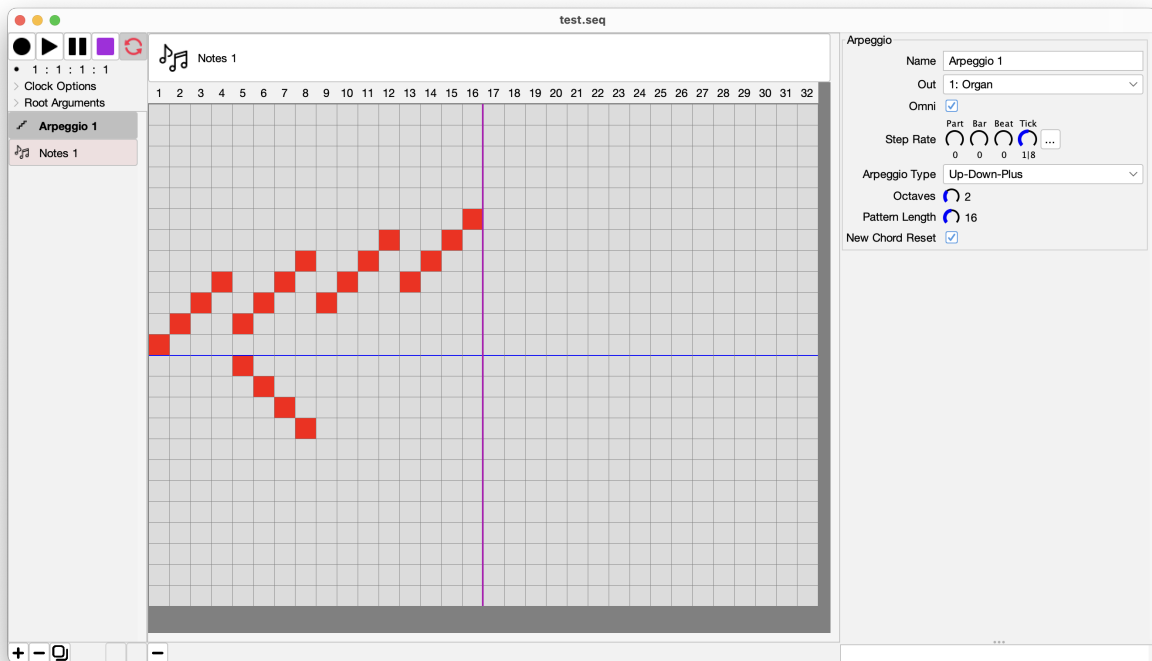


Figure 12 Arpeggio Motif.

2.8 Arpeggio

Arpeggio doesn't make any sound on its own, nor does it control the order or time of play of its children. Rather, it is a *MIDI Effect*: it modifies the MIDI it receives from its (only) child. Specifically, it interprets the MIDI coming from its child as chords, and replaces them with arpeggios, just like an arpeggiator on a synthesizer.

Source of Chords and Emitting MIDI To use Arpeggio, you have to provide it with a source of chords. Usually this will be a **Notes** motif, or perhaps something that contains Notes motifs as children. You drag your chord source from the Motif List into the single slot at the top of the Arpeggio.

When a chord source like Notes emits MIDI, it does so out some **MIDI Device**. You can instruct Arpeggio to intercept *all* MIDI Note data from its child, and use it to make arpeggios, by checking **Omni** in the Inspector. Otherwise Arpeggio will only intercept MIDI Note data whose device is the same as **Out** (set in the Inspector).

Arpeggio itself emits MIDI in the form of arpeggios. **Out** specifies the MIDI output device that it will send its arpeggios to.

Step Rate The Step Rate governs the speed of the arpeggio being played. An arpeggio is a sequence of notes, each played in turn for the same amount of time. The amount of time of a single not (or *step*) is specified by the **Step Rate**. By default this value is a 16th note. Don't set the Step Rate to 0, which is impossible: it will instead assume you meant 1 (which is ridiculously fast).

Arpeggio Types Arpeggio can make many kinds of arpeggios, which you specify with **Arpeggio Type** in its Inspector:

- **Up** Arpeggiates the chord upwards.
- **Down** Arpeggiates the chord downwards.
- **Up-Down** Arpeggiates the chord upwards, then downwards but without the top note. For example, if Arpeggio was provided the chord C E G A, it would repeat the arpeggio C, E, G, A, G, E.
- **Up-Down-Plus** Arpeggiates the chord upwards, then downwards with the root note repeated at top. For example, if Arpeggio was provided the chord C E G A, it would repeat the arpeggio C, E, G, A, C2, A, G, E. (Where C2 was one octave above C). This is more useful than **Up-Down** for arpeggios with multiple octaves (see below).
- **Random** Arpeggiates the chord by playing its notes in random order. Random attempts to not play the same note twice when reasonable.
- **Pattern** Arpeggiates the chord according to the provided pattern. See **Patterns** below. The **Octaves** option is ignored.

An arpeggio is played for some number of **Octaves**. For example, if you set Octaves to 2, and your chord is C E G A, and you're playing an Up arpeggio, it'd repeat C E G A C2 E2 A2 G2, where C2, E2, A2, and G2 are one octave above C, E, G, and A respectively.

Pattern Arpeggios Arpeggios get fun when you start drawing patterns in the grid at the center of the window. Each column is a step: you can have patterns up to 32 steps long, but you probably will want a shorter pattern: set the length of your pattern, in steps, with the **Pattern Length** dial in the Inspector. You'll see a purple line mark the end of the pattern.

You instruct the Arpeggio to play a note at a given step by clicking on a square to highlight it.

The blue line is the *Root Marker*. The square directly above it represents the root note in whatever chord is currently provided. The square above *that* represents the second note in the chord, and so on. When you run out of notes, the next squares represent the notes in the chord repeated again, but played one octave higher.

For example, if your incoming chord was C E G A, then the first four squares in a column, above the root marker, would represent C, E, G, and A. The next four squares would represent C1, E1, G1, and A1 (one octave above). The next would represent C2, D2, G2, and A2, and so on.

You can play notes *below* the chord as well! The squares *below* the Root Marker represent notes one or more octaves below the chord. For example, the first four squares below the Root Marker would be (going down) A -1, G -1, E -1, and C -1, which are all one octave below the chord. The next four squares below them would be A -2, G -2, E -2, and C -2, and so on.

You can play more than one note in the same step: just click multiple squares. You can also play a **rest**: just have a column with no squares selected in it. Finally, you can make **ties** to extend the length of notes. If you shift-click (or right-click) on a pattern note, you will make a tie. A tie indicates that the immediately previous note will be lengthened and continued. If there is no immediately previous note (it's a rest), then the tie has no effect. Ties are blue.

Velocity By default, the velocities (volumes) of the arpeggiated notes are based on the velocities of the underlying notes provided Arpeggio. If you uncheck **As Played** in the Inspector, then you can fix the velocity of all the arpeggiated notes to a given value by setting the **Velocity** dial.

Resetting Arpeggios When the underlying child says it has finished, Arpeggio will tell the same to its parent. Arpeggios are always reset when the parent resets or loops the Arpeggio. Also by default Arpeggios are reset when *all* the old notes are released and entirely new ones are played. As long as at least one note is being played, the Arpeggio won't reset.

If you want to override this chord reset behavior, so that Arpeggio doesn't reset even when all notes are released, just uncheck **New Chord Reset**.

Activity Arpeggio is a filter: its main job is not to control how and when its child plays, but rather to process the MIDI received from its child. You might wish to say *when* Arpeggio produces arpeggios from its child — that is, when it is **active**. To do this, uncheck the **Always** checkbox (which says that Arpeggio is always active no matter what), and set the activity time range by changing **From** and **To**.

When Arpeggio is *inactive* it just passes its child's MIDI straight through to its parent, rather than intercepting it to do arpeggios.

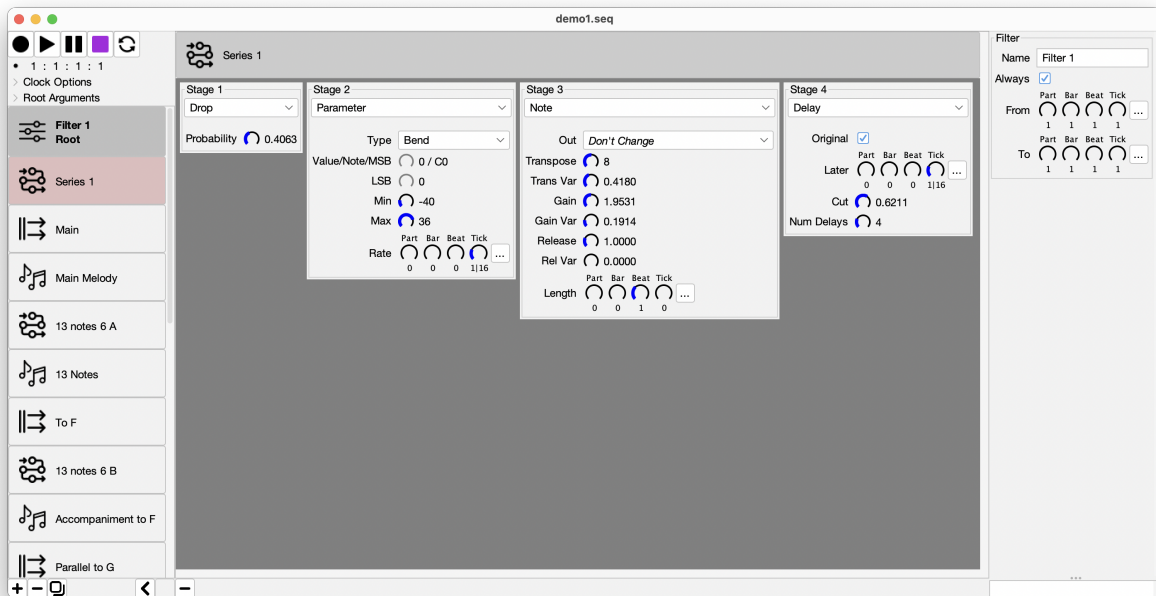


Figure 13 Filter Motif.

2.9 Filter

Just like Arpeggio, Filter doesn't make any sound on its own, nor does it control the order or time of play of its children. Rather, it applies a series of *MIDI Effects* which modify the MIDI received from Filter's (only) child. Filter has many different ways of adding to, removing, or modifying this MIDI data.

Filter lets you choose a MIDI Effect for each of (at present) four **stages**. When it receives MIDI data, it hands it off to the first stage, which modifies it as it sees fit. Then it is handed to the second stage for further modification, and so on. When all the stages have completed modification, Filter hands the resulting MIDI data to its parent, and ultimately to be emitted by Seq.

Activity Filter's MIDI Effects are only active for a certain range of time. Outside that time, MIDI is not modified, but just passed through. Normally this range of time is *all the time*, because **Always** is checked in the Inspector. If you uncheck this checkbox, you can set the activity time range by changing **From** and **To**.

Filter's MIDI Effects can also be restricted to a specific Output device, which you can set with **Only**. By default they operate on **All** Output devices. Note that some MIDI Effects change the Output device of various MIDI data. Filter decides which MIDI data will be filtered (via **Only**) well before this happens, so the data will be continued to filtered regardless of what Effects change it to.

MIDI Effects At present, Filter has six MIDI Effects available:

- **None** Doesn't change the MIDI at all.
- **Note** Changes note pitch, velocity, release velocity, and length.
- **Delay** Delays the onset of a note, or repeats it multiple times with a delay in-between, or repeats it multiple times simultaneously (as in unison).
- **Drop** With some probability, drops a note (don't let it pass through), or drops events within an interval of time.

- **Noise** Adds noise to the value of a non-note parameter such as pitch bend, CC, or aftertouch.
- **Map** Maps a non-note parameter value through a function to transform it to a different value.
- **Scale** Quantizes incoming notes, forcing them to conform to a certain scale or notes in a chord.
- **Chord** Changes an incoming note into a set of notes forming a chord or interval.

We'll describe these next in detail. More MIDI effects are planned for the future.

At the end of this list is **Copy From...**, where you can ask that a MIDI Effect in the given stage, and all of its parameters, be copied exactly from another stage.

The Note Effect This effect allows you to change features of the notes being passed in:

- **Out** You can change the MIDI output device.
- **Non-Note Out** You change the MIDI output device for all non-note events in addition to notes.
- **Transpose and Trans Var** You can transpose all notes, and can also add random transpositions to them. The variance ranges from 0 to ± 2 octaves.
- **Gain and Gain Var** You can multiply the velocity (thus velocity) of all notes by a given gain, and can also multiply a random random gain to them. The variance ranges from $0\times$ to $4\times$.
- **Release and Release Var** You can multiply the *release velocity* of all notes by a given gain, and can also multiply a random random gain to them. The variance ranges from $0\times$ to $4\times$.
- **Change Length and Length** You can set the length of all notes.
- **Adds to Length** If checked, then you can *add to* the length of notes rather than *set* their length. Note that **Change Length** must still be checked.

The Delay Effect This effect adds delays and unison to notes. You can add one or more delays to an original note, and choose whether to play the original at all. If you set the delay interval to zero, you can bulk up your original note with more of the same note, creating a **unison** effect.

The Delay options are:

- **Original** Whether to play the original note at its original time.
- **Interval** The interval between successive delays.
- **Cut** How much to multiply the current delay volume by to get the volume of the next delay. This eventually will cut delays down to 0 volume.
- **Num Delays** How many delays to do (besides the original, if you're playing that one).
- **Random** Whether the delay is fixed or random. If random, then the actual delay will be between 0 and Interval in length. This is particularly useful to humanize notes by adding a slightly randomized delay to when they're actually played. To do this, set Original to false, set Cut to full, set Num Delays to 1, set Random to true, and then set Interval to the maximum delay length you'd want to add (usually something very small, perhaps 40 ticks). Note that you cannot cause notes to be played slightly *earlier*, just *later*: Delay doesn't know what notes are arriving.

MIDI delays can wreak havoc on some simpleminded synthesizers. MIDI instructs synthesizers to start playing a note with a *Note On* message (specifying the pitch), and to stop playing the note with a *Note Off* message (with the same pitch). Imagine that you have a single short delay on a long note. The note commences with a *Note On*, and then the Delay issues a *Note On* (for the *same pitch*) for the delayed note. Then the original note ends with a *Note Off*, followed by another *Note Off* for the delay. But some synthesizers can only play *one note for a given pitch at a time*, so when the second *Note On* arrives, the first note is unceremoniously cut short to make way for the new one. Or when the first *Note Off* arrives a synthesizer may not know which of the two notes to turn off; or it may turn off both notes, and so on. Finally, a large delay count may overwhelm the total number of notes a synthesizer can play at one time, causing it to take drastic measures to compensate.

If your notes are short enough, and have a long enough time between them, such that delays never overlap with a note of the same pitch, then you'll probably be fine. Otherwise, expect Delay to present challenges to your synthesizer.

The Drop Effect This effect randomly deletes notes, and also deletes all notes and non-note events within a certain region.

- **Delete All** Whether we should delete all notes and non-note events.
- **Note Probability** If **Cut** is unchecked, then the probability that a note should be deleted at random.

The Noise Effect This effect adds noise to the values of non-note parameters. Every so often this effect chooses a new random value, then consistently adds that value to all the parameters that arrive of the given type.

- **Type** The parameter type in question to add noise to. The options are *Pitch Bend*, *Control Change* (CC), *Non-Registered Parameter Numbers* (NRPN), *Registered Parameter Numbers* (RPN), and *Aftertouch*.
- **Param/MSB** The parameter to add noise to. Pitch Bend doesn't have a parameter. Aftertouch also doesn't have a parameter (note is disregarded — all aftertouch messages are treated the same). For NRPN and RPN, this serves as the *Most Significant Byte* (MSB) of the parameter to add noise to.
- **LSB** The *Least Significant Byte* of the parameter to add noise to. This is only used by NRPN and RPN, and complements the MSB.
- **Variance** The noise variance, from 0.0 to 1.0. The highest noise variance adds extreme noise to the parameter.
- **Rate** How often the parameter noise value is updated.

The Map Effect This effect maps the values of a non-note parameter to different parameters. It does this as follows:⁴

1. It takes the value and bounds it between a min and a max.
2. It maps the range min...max to 0...1, so the value is now between 0 and 1.
3. It pushes the value through a function, for example, squaring the value. The input and output are always between 0 and 1.
4. It then maps from 0...1 back to min...max, so the value is now back in its normal range.

⁴Note that if the parameter is pitch bend, and the value is negative, it is first made positive before any of this; then the result is made negative again at the end.

5. It emits the final mapped value.

This allows you to change parameter values in many ways. You could invert them, add an amount to them, square them or invert-square them (kind of like a square root), etc.

- **Type** The parameter type in question to add noise to. The options are *Pitch Bend*, *Control Change* (CC), *Non-Registered Parameter Numbers* (NRPN), *Registered Parameter Numbers* (RPN), and *Aftertouch*.
- **Param/MSB** The parameter to add noise to. Pitch Bend doesn't have a parameter. Aftertouch also doesn't have a parameter (note is disregarded — all aftertouch messages are treated the same). For NRPN and RPN, this serves as the *Most Significant Byte* (MSB) of the parameter to add noise to.
- **LSB** The *Least Significant Byte* of the parameter to add noise to. This is only used by NRPN and RPN, and complements the MSB.
- **Map** The mapping function used. See below.
- **Min** The minimum of the bounding range.
- **Max** The maximum of the bounding range.
- **By** A value used by some mapping functions to determine how strong their effect is.

The mapping functions are as follows. Let *param* be the parameter and *by* be the value of the **By** dial. Both of these values range from 0 to 1. The result of the mapping function is bounded to be within 0 and 1.

- **None** *param* is not modified at all.
- $param \leftarrow by - param$. This is particularly useful if $by = 1$: this just inverts *param*, making high values low and vice versa.
- $param \leftarrow param + by$.
- $param \leftarrow param - by$.
- $param \leftarrow param \times by$. This sounds like it makes *param* bigger but in fact it makes it smaller, since $by \leq 1$.
- Discretize *param* by *by*. This creates a staircase effect. Here, *by* is treated as a value from 1 to 16. *param* is set to the nearest division of 1.0 into *by* chunks. For example, if $by = 8$. and *param* is 0.352, then *param* is rounded up to 0.375 (that is, $3/8$).
- Discretize *param* by *by*, and subtract from 1 (inverting it).
- $param^2$. This makes *param* rise more slowly and in a curved fashion.
- $param^4$. This makes *param* rise even more slowly and in a curved fashion.
- $1 - (1 - param)^2$. This makes *param* rise faster and in a curved fashion.
- $1 - (1 - param)^2$. This makes *param* rise even faster and in a curved fashion: it's much like the exponential rise in an envelope.

The Scale Effect This effect forces incoming notes to quantize to the nearest note in a scale or chord.

- **Key** The key of the scale or chord.
- **Round** How to force the incoming notes. You can **round down** the note to the nearest note in the scale/chord equal to or below the incoming note. Or you can **round up** to the nearest note equal to or above the incoming note. Or you can round the incoming note to the **nearest** note in the scale/chord: if you do this, ties are broken by rounding **up** or **down**.
- **The Notes** You can select which notes in the scale or chord you want to force the note to conform to. A variety of **Presets** are also offered.

The Chord Effect This effect converts a note into a full chord or interval. It is similar to the classic *chord memory* effect in certain synthesizers.

- **Chord/Interval** The chord or interval to convert the note to.

Parameterization Many Effect features can be **Parameterized** by double-clicking on the dials. For more on parameterization, see Section 4, (*Parameterized Control of MIDI Data*).

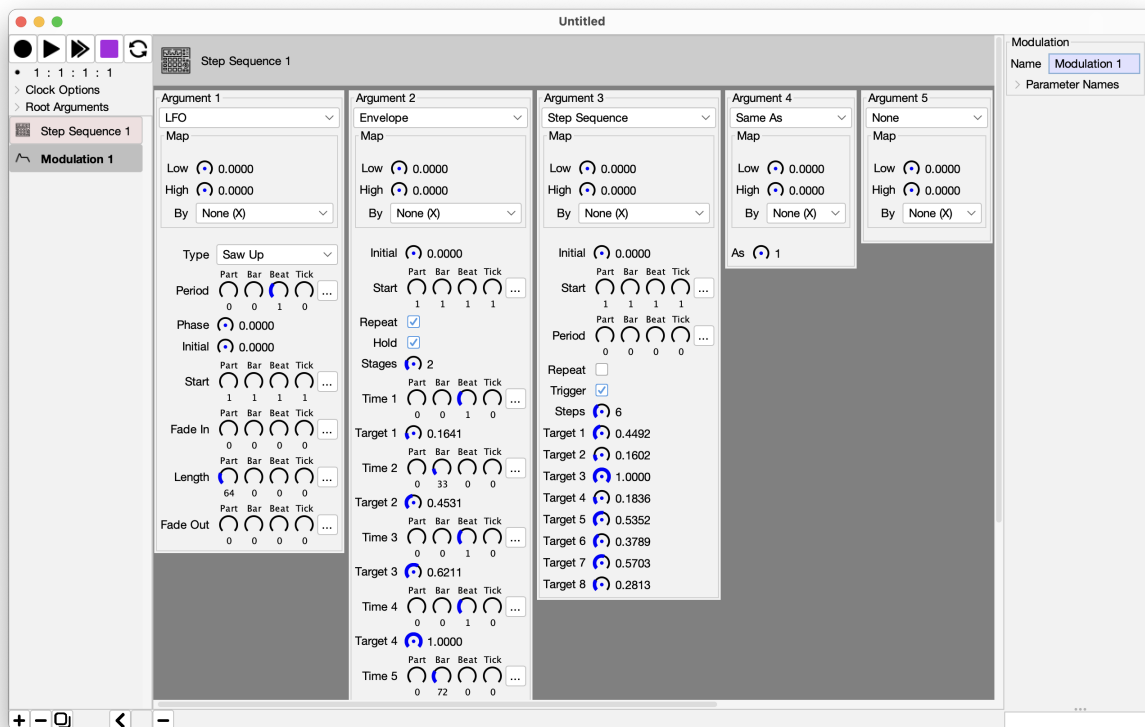


Figure 14 Modulation Motif.

2.10 Modulation

Modulation doesn't make any sound on its own, but it's not a MIDI filter. Rather, it take a single child and provides some ways for you to modify the child's **parameters**.

Note that, like Filter and Arpeggio, Modulation requires a single child to operate on, or else it does nothing!

Before we go any further, it might be helpful for you to read Section 4, which discusses parameters. In short: each motif has eight parameters, each of which can be set to any value from 0.0 to 1.0. Many dials in a motif can be the value of one of the motif's parameters. Furthermore, in the child's parent, you can modify the parameters of a child (from a parent's perspective, a child's parameters are called its **arguments**). If you don't, by default the settings of a motif's parameters are automatically set its parent's own parameters. This all allows you to change one or more dial settings in a child, or a deep descendant, or many of children or descendants at the same time, by setting a parent or ancestor's parameter values.

In most motifs you can directly set a child's arguments to values as you see fit, or to random values. And Select lets you set its own parameters via big Dials, or via MIDI CC messages (Seq's root parameters can also be set via MIDI CC messages). But **Modulation** goes much further: it lets you change the parameters of a child via an automated modulation procedure such as a Low Frequency Oscillator (LFO), Envelope Generator, or a simple parameter Step Sequencer. This gives you a way to change the parameters of a child automatically over time.

Mapping After the parameter value is determined by the automatic modulator you have chosen, it is passed through a **mapping function** before it is finally set. This allows you to scale the value to within a range of your choosing, invert it, and warp it using certain warping functions.

Your parameter value, output by the modulation function, has a range of 0.0 to 1.0. We first warp it, using one of the following functions:

- **None** (X) No warping occurs.
- X^2 The argument is squared.
- X^4 The argument is squared twice.
- $1 - (1 - X)^2$ The argument is inverted, the squared, then inverted again. This looks a little like subjecting it to a square root.
- $1 - (1 - X)^4$ The argument is inverted, the squared twice, then inverted again. This looks a little like subjecting it to a square root twice.

After warping, the parameter still has a range from 0.0 to 1.0. We then scale it to a new range from **Low** to **High**. Note that **Low** can be *higher than High*: in this case the parameter range is inverted.

After all this mapping, the parameter is then passed to the Child.

Modulation Functions At present, Modulation has five automated modulation functions available:

- **None** No automation.
- **LFO** The argument is modulated over time using a **low-frequency oscillator**, which produces a repeating function.
- **Envelope** The argument is modulated over time using an **envelope** with up to eight stages.
- **Step** The argument is modulated over time using a **parameter step sequencer** with up to eight stages.
- **Constant** The argument is fixed to a constant value.
- **Same** The argument is set to the same value as some earlier modulation function.

We'll describe these next in detail. More modulators are planned for the future.

At the end of this list is **Copy From...**, where you can ask that both the Modulation Function and the Mapping for a given Argument be copied exactly from another Argument.

None No modulation occurs. The value of argument X in the child is set to the equivalent parameter X in the Modulation parent. However, the argument is optionally mapped first.

LFO The value of argument X is set to the output of an LFO. The LFO has the following parameters:

- **Type** The LFO wave. This can be Saw Up, Saw Down, Square, Triangle, Sine, Random, and S&H (Random Sample and Hold). Random works by selecting a random target position from 0.0 to 1.0 and slowly moving linearly to reach that position by the end of the period, at which it picks a new random target position and repeats. S&H works similarly, but it doesn't move: it just holds at its previous target position until the period is completed, at which point it jumps to the new target position.
- **Period** The length of one repetition of the wave, in time steps.
- **Phase / Var** For the **Random** and **Sample and Hold (S&H)** wave types, this defines the **Variance**, that is, how wide a swing in randomness is used when computing the next target location. For all other wavetypes, this defines the **initial phase** of the wave, ranging from 0.0 to 1.0.

- **Initial** The initial value of the LFO. The LFO outputs this until it reaches its **Start** time.
- **Start** The LFO's start time. The LFO doesn't begin operating until this time: instead it sets the argument to **Initial**.
- **Fade In** After the LFO's start time, the LFO gradually fades in from the initial value to the full range from 0.0 to 10. This is done over the Fade In period: this is a time interval, not a specific time step.
- **Length** After the fade in has completed, the LFO will run for the Length. This is a time interval, not a specific time step.
- **Fade Out** After the length has completed, the LFO will fade back out to the Initial value over this period. This is a time interval, not a specific time step. After the Fade Out is completed, the LFO will simply set the argument to the Initial value again.

LFO can also be used to randomize a parameter at a constant rate (unlike the *random parameter*, which is only set when the motif is reset or looped). Set the type to Random, the period to your desired rate, and the mapping min and max to your desired random bounds.

Envelope The value of argument X is set to the output of an up to eight stage envelope with the following parameters:

- **Initial** The initial value of the envelope. The envelope outputs this until it reaches its **Start** time.
- **Start** The envelope's start time. The envelope doesn't begin operating until this time: instead it sets the argument to **Initial**.
- **Stages** The number of stages in the envelope.
- **Time n** The length of time for stage n . This is a time interval, not a specific time step.
- **Target n** The target value for stage n . During its time interval, the envelope will gradually move from the previous target value (or from the initial value) to this new target value.
- **Repeat n** When checked, the envelope will loop: when it finishes its last stage, it will go back to its first stage and continue from there.
- **Hold n** When checked, the envelope will act like a **Sample and Hold**: it will not gradually move towards its target value, but rather will stay at its previous target value (or at the initial value) until the stage is completed, at which time it will suddenly jump to the new target value..

Step Sequence This is much like a simplified Envelope with Hold: where each stage has the same duration. We call these stages **steps**.

- **Initial** The initial value of the step sequencer. The step sequencer outputs this until it reaches its **Start** time.
- **Start** The step sequencer's start time. The step sequencer doesn't begin operating until this time: instead it sets the argument to **Initial**.
- **Step Length** The length of a single step. This is a time interval, not a specific time step.
- **Steps** The number of steps in the envelope.
- **Step n** The value of Step n . When in Step n , the argument is set to this value.

- **Repeat n** When checked, the step sequencer will loop: when it finishes its last step, it will go back to its first step and continue from there.
- **Trigger n** When checked, the step sequence will not output the given when it reaches Step n . Rather, if the value is non-zero it will issue a short one-time-step burst of 1.0, then output 0.0 for the rest of the step time. If the value is zero, it will always output 0.0 for the step time. This is a **trigger**: a short burst of 1.0 to indicate a trigger has occurred along the argument. See the **Series** motif for an example use of a trigger.

Constant This indicates that parameter is fixed to a certain **constant value**. Mapping still occurs, but it's not very useful.

Same This indicates that the modulation function for this argument (let's call it m) is the same as the one for some other argument n . Note that n must be set to **less than m** , or Same will indicate that the value is **Invalid**, and will act just like **None**. Mapping still occurs regardless: thus Same can have a different mapping from the Node that it is the Same as.

Combining Modulations If you have a Modulation set up as a child to a Modulation, you can combine modulations on the same parameter. For example, let's say you wanted to do an LFO until an envelope started, and when the envelope finished, you continued the LFO. You create an LFO in the parent Modulation for parameter X , going all the time. In the child Modulation for parameter X , you create an envelope which is delayed to start a while later. Set the Initial value for the child Modulation to be *Param X* rather than a fixed number. Add an additional stage at the end of the envelope. Set its target value to be *Param X* as well. Finally, set the time for the additional stage to be however long you'd like to fade from the envelope's termination to the LFO again.

You can do tricks like this for any combination of modulation methods.

Parameterization As usual, many Modulation features can be **Parameterized** by double-clicking on the dials. For more on parameterization, see Section 4, (*Parameterized Control of MIDI Data*).

Child Repeats As is the case for Serial, you can specify the number of times the child will repeat (loop) beyond its initial iteration before the Modulation determines that it has finished.

Child MIDI Like a number of other motifs, Modulation lets you modulate the rate, transpose, gain, and out of your (sole) child. There is a difference however. If you set the rate, transpose, or gain to a *parameter*, keep in mind that the whole point of Modulation is to modulate *its own parameters*, which are normally passed down to the child. However these parameters that Modulation is modulating will *also* affect your Child MIDI settings. This is by design: if for example you set your Child's MIDI gain to Param 1, then if you set up an Envelope on Argument 1, it will modulate that gain!



Figure 15 A Macro with Two Macro Children.

3 Macros and Macro Children

A **Macro** is a whole Seq sequence DAG bundled up into a single Motif. When the Motif is played, the underlying sequence is played until it is finished.

To create a basic Macro, you first save your sequence. (Remember that the sequence is saved out starting at the Motif Root, ignoring its parents and other non-descendant Motifs.) Then you add a Macro to the Motif List and when you do it will prompt you to load that sequence into the Macro. When you play the Macro, its stored sequence is played as-is.

This has some usefulness, but Macro can go further than this. You can add special Motifs to your sequence called **Macro Children**. A Macro Child has a single Inspector feature: its **Name**: come up with a good name for it. If you have Macro Children in the sequence you have saved out, they become temporary placeholders for actual Children of the Macro in your main sequence.

This means that if you had three Macro Children nodes in your macro's internal sequence for example, then the Macro itself will take up to three Children. As the Macro is playing, it will reach its Macro Children nodes, and instead of playing them, the Macro will play the corresponding Child attached to it. Figure 15 shows a Macro loaded that had two Macro Children nodes inside, and thus has two slots for Children, which we have populated with a Notes and a Series from the Macro List respectively.

What's the point of this? It allows you to use Macros to create patterns or templates, and then populate the templates with different Children as you see fit later on.

The Child slots of a Macro are populated in the usual way: by dragging from the Motif List. You can also clear a slot by pressing the **Delete Button** below. You can change the Name of the underlying Macro Child node: this is now the **nickname** of the Child in its Inspector.

4 Parameterization

Many of the dials on Seq's motifs are **Parameterizable**. This means that instead of setting a dial to a value, you can arrange for it to be changed elsewhere in Seq by something else (another dial, a MIDI message, an automatic mechanism). This makes possible many things. For example, you could:

- Control a dial from a remote MIDI CC.
- Control multiple dials in a motif — for example, the velocity of four different notes — together from the same dial elsewhere.
- Control several dials in different motifs — for example, the swing of three different Step Sequencers — together from the same dial elsewhere.
- Set a dial to a random value before its motif begins playing.
- Control dials dynamically from an automatic mechanism such as an LFO or an envelope.
- Send a **trigger** to a motif, such as a telling a Series to stop repeating one of its children.

Parameters Each Motif has eight **Parameters**. Just before the Motif's parent steps the Motif to advance it a little bit, the parent has the opportunity to change the values of any of those Parameters. The parent can change them to any value from 0.0 to 1.0, or it can set them to any of the current values of the parent's *own Parameters*. (In fact, by default the value of Parameter X in a Motif is set to the value of Parameter X in its parent).

Normally changing a motif's parameter does nothing. But many of the dials you see in a Motif can be **bound** to one of those parameters, meaning that if that parameter is changed, the dial will also be changed. This allows parents, or ancestors, to change one or more dials in their children or descendants. Multiple dials can be bound to the same parameter, meaning that by changing a single parameter, a parent can simultaneously affect a number of Dials, even spread across multiple child Motifs.

Arguments From the child motif's perspective, a parameter — the thing it may have connected its dials to — is unsurprisingly called a **parameter**. But from the parent's perspective, changing or setting one of its child's parameters is called changing an **argument** in the child. This terminology is borrowed from computer science, because to a computer programmer, motifs may be thought of as functions (or methods or procedures), and functions have arguments (set by the caller of the function) and parameters (read inside the function).

Binding a Dial to a Parameter You can tell that a dial can be bound to a parameter because it has a little red or blue **dot** in the middle of it. You bind the dial by double-clicking on it. This brings up a pop-up menu as shown in Figure 16 at right. Here you can bind the dial to any of its motif's parameters, here shown as **Param 1 ... Param 8**, or to the motif's random parameter (**rand**).

After you have bound a dial to a parameter, the dial's will go gray, the dot will go red, and the name of the parameter will be shown along side it. If you would like to break the binding and go back to setting the dial to a fixed value, just turn the knob.

Changing the Parameter Name Don't like the name **Param 1**? Want something more descriptive so you can remember what the heck that parameter is for? Many motif inspectors have a **Parameter Names** section where you can change the names of all eight of the motif's parameters. You will have to click on the Parameter Names to open it up.

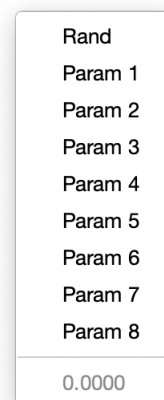


Figure 16 A Parameter Menu

Changing a Child's Argument Values from the Parent Many parent motifs can change the arguments of their children. If you click on a child motif, its inspector may have a region called **Arguments** (you will have to click on it to open it up). You can see the Arguments list in Figure 17 at right. Here you can set the values of the arguments sent to the child's parameters. You can also set the minimum and maximum bounds of the child's random parameter.

By default the arguments are called **Arg 1 ... Arg 8**. But if you provide names for the child's parameters, they will appear here instead.

Note that these dials are themselves parameterizable: this means you can bind the child's parameters to the parent's parameters (and so on up the chain). In fact by default the arguments are bound to the parent's parameters already. For example, **Arg 1** is set to the parent's **Param 1** and so on.

Not all parents can change their child parameters even though they have children. Notably the Arpeggio, Modulation, and Filter motifs cannot change the parameters of their sole child. Instead the child's parameters are simply set to their own parameters.

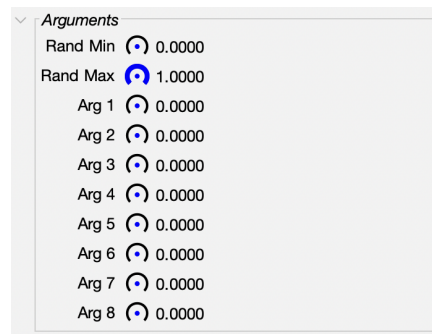


Figure 17 Arguments List.

The Random Parameter In addition to the eight parameters, the Motif also has an additional **Random Parameter**. This parameter is set to a random value once each time a Motif is started or looped. This value is not changed while the Motif is stepped. Dials can also be bound to this value. Additionally, a child's arguments can be bound to the parent's own random value.

A parent cannot change the value of its child's random parameter: it is set to a random value. But the parent can change the **minimum and maximum bounds** from which the random value is selected. These bounds, like any other parameter, can be set to fixed values or bound to the parent's own parameters or to its random parameter.

A Warning about Child Arguments Consider the Series motif. Each of its children can have (for example) Initial Repeats. These can be bound to parameters. But note that these features are *not features of the Child*: they are features of the *parent Series*, and thus they are bound to the Series's parameters, not to the Child parameters. This can be a little confusing. In short, if a feature appears anywhere in *any Inspector* associated with a Motif (the Motif Inspector, one of its Child inspectors, etc.), that feature *belongs to the Motif* when it comes to parameterization. It doesn't belong to the Child.

Triggers The Series and Automaton motifs have special options to repeat children forever until receiving a **trigger** on Parameter 8. What is a trigger? A trigger on Parameter 8 is, well, triggered, when Parameter 8 changes from a value < 0.5 to a value ≥ 0.5 .

A parameter is just a real-valued number between 0.0 and 1.0. But of course many Seq dials don't go 0.0 to 1.0: they go 0...127 or 0...4 or whatnot. These ranges are mapped as appropriate. This can make setting the parameter a pain, but you can tell what the appropriate mapping for a parameterizable dial is but double-clicking on it: at the bottom of the pop-up menu is a grayed-out value. That's the real-valued parameter value that would correspond to the current feature value.

The Root Arguments If the Root Motif has bound some dials to its parameters, where are those arguments set? They're set in the Root Arguments, located below the Transport (the play and record buttons, etc.) See Figure 18 at right. This is also where you can change the **Random Seed** of your sequence. If you change the random seed, the random number generator will produce a different sequence of numbers next time.

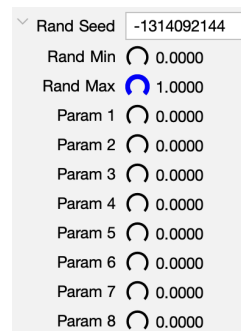


Figure 18 Root Arguments with default settings.

For each root argument, you can also set a **Control Change** or **CC** parameter to allow an external controller to change the argument in real-time as you see fit. To do this, you'll need to specify the **CC In** device from which these CC messages will come. When you start playing a sequence, the root arguments will be set to the fixed values you specified: but if you afterwards send CC messages, they will be changed. If you don't want to use this feature, set **CC In** to *None*: it's a bit more efficient.

Macro Root Arguments You'll note a dot in the center of each of the root arguments, and you thus by double-clicking on it you can change each argument to say *Macro*. This only has an effect once the root is saved out as a macro: otherwise, setting the argument to *Macro* is the same as setting it to 0.

When you save out a macro, the parameters of the macro's root can be set to fixed values (the root arguments) or they can be set to *Macro*. This means that the root's parameter will be set to whatever the Macro's parameter values are set to when you use the Macro from its parents in the outer sequence. For example, if Root Parameter 3 is set to *Macro*, then when used in a Macro, Root Parameter 3 will be bound to the Macro's own Parameter 3. Thus you can either fix the parameters to values or you can allow them to be customized by the Macro's parents.

How Parameterization Works To understand how parameterization is updated, let's first make sure we understand how parent motif instructs a child to play. First, before any playing, the parent **resets** its child to set its time position to 0 and to initialize it. Then each timestep (192 times per beat), when the parent itself is being **pulsed** by *its parent*, it in turn pulses the child to advance it. At some point the child will tell its parent that it has **finished**, or for some other reason the parent decides to stop playing the child. At this point the parent will **stop** the child — it will tell the child that the playing has ended. Then the parent can either go about doing other things, or it can **loop back** and replay the child as it sees fit. If it replays the child, it once again resets it and starts the process again. So in short, it does:

Woven in-between these steps are opportunities for the parent to set parameters in the child. Each motif has eight parameters that its parent can change. Additionally, the motif has a **random value** which is selected (at random!) when the child starts playing: the parent does not set this directly, but it can change the minimum and maximum values from which the random value is chosen. Adding the parameter-setting stages, we have:

1. Before playing, reset the child.
2. Before playing, the child generates its random value.
3. Each time the parent is pulsed, as it sees fit:
 - (a) Change the child's parameters as it sees fit.
 - (b) Pulse the child
4. After all playing has finished, stop the child.
5. Either continue doing something else, or loop back to step 1 as it sees fit.

Note that looping back will change the random value to a new number.

What Motifs are Parameterizable?

- **Notes** and **Step Sequence** have dials that can be set to parameters, and you can name the parameters.
- **Series**, **Parallel**, **Select**, **Filter**, **Modulation** and **Automaton** have dials that can be set to parameters, and you can name the parameters. Furthermore these motifs have children, and you can change each child's arguments.

- The parameters of **Silence** don't do anything, because these motifs have no parameterizable dials. You can't name the parameters for the same reason.
- The parameters of **Macro Child** are simply passed down to the Motif assigned to the Macro Child when you are using its Macro. You can't name the parameters.
- **Arpeggio** has no parameterizable dials. Its parameters are simply set to pass data down to the parameters of its sole child. You can't name them: their names will be the names of the parameters of its sole child.
- **Macro** has no parameterizable dials. When you change the arguments of a Macro, these values are passed down to the root Motif of its internal sequence. Alternatively you can set Macro to always send the same fixed values to its root Motif, ignoring the arguments it receives. This choice is made per-argument when you save the Macro out. See **Root Macro Parameters** above.

4.1 Parameterized Control of MIDI Data in Series

The **Series** motif has a trick up its sleeve: the ability to output parameters as MIDI CC, NRPN, RPN, Pitch Bend, or Aftertouch. You can find this by clicking on **MIDI Parameters** in the Series inspector. This will expand to **Param 1** through **8**, along with **Out**. For each Param you can select a Type (such as CC or NRPN), and then a number if appropriate.

Let's do an example. First set Param 1's Type to CC. This reveals a Dial you can set to specify the CC number (from 0–127). Let's say we set this to 5. Finally let's set Out to the output device which will receive the CC MIDI messages.

Now, whenever the Series is playing, if its incoming Parameter 1 has changed in value, it will be mapped to a CC value and emitted out CC 5. Parameters have values from 0.0 to 1.0, whereas CC values range from 0 to 127. So for example if Parameter 1 changes to 0.5, this will be mapped to 63, and so Seq will emit a CC message number 5 with a value of 63.

The rate at which MIDI data is output is chosen by the **Rate** parameter. MIDI data will only be output if the parameter has changed since the last time MIDI data was output.

The following MIDI output types are available for each Parameter.

- **None** Nothing is emitted.
- **CC (Control Change)** You specify the CC number (0–127). Incoming parameters will be mapped to CC values (also 0–127).
- **14-bit CC** You specify the CC parameter (0–31). Incoming parameters will be mapped to 14-bit CC values (ranging 0–16383).
- **NRPN** You specify the NRPN parameter (0–16383) with two dials: the left one specifies the coarse value or MSB, and the right one specifies the LSB. The final NRPN number is displayed. Incoming parameters will be mapped to NRPN values (also 0–16383).
- **Coarse NRPN** You specify the NRPN parameter (0–16383) with two dials: the left one specifies the coarse value or MSB, and the right one specifies the LSB. The final NRPN number is displayed. Incoming parameters will be mapped to NRPN coarse (or MSB-only) values (0–127).
- **RPN** You specify the RPN parameter (0–16383) with two dials: the left one specifies the coarse value or MSB, and the right one specifies the LSB. The final RPN number is displayed. Incoming parameters will be mapped to RPN values (also 0–16383).
- **Pitch Bend** Incoming parameters will be mapped to Pitch Bend values (–8192–8191). Thus if you set the Parameter to 0.5, then this will map to 0.
- **Channel Aftertouch** Incoming parameters will be mapped to Channel Aftertouch values (0–127).

4.2 Select's Overriding of Parameters

Select has a performance option where it can override its own parameters, setting them to values chosen by Select and not by its parent. Why would you want to do this? Because the parameters of various children of Select can be bound to those parameters. This allows you to change the parameters of children in real-time via a Big Dial, a Joystick, or a MIDI CC message from a remote controller.

Overriding a Parameter In the **Dial Parameters** region of Select's Inspector, check the box of the parameter you'd like to override (Dial 1 through Dial 8, corresponding to Parameters 1 through 8). When you do so, note that the grid will change to reveal the Big Dial for that parameter.

Next, select a child, and in the Child Inspector, go to the Arguments section. In there, change the corresponding Argument to be the Parameter. For example, if you overrode Dial 3, and you want this to modify the Child's Argument 2, you'd set Arg 2 to Param 3.

Now, when the Select is playing, you can change the value of this parameter of Dial 3 (for example), and it will update the Child's Parameter 2 to reflect this value in real-time.

You can set up multiple children this way to respond to Dial 3.

In fact, by default all children's parameters are just set to Select's parameters. This means that if you turn on and change Dial 3, it will affect parameter 3 of every child. So in most cases if you want to set a parameter, you're probably already set unless you want to change which parameter it is bound to. In fact in most cases you might only want to *disconnect* a parameter from some child.

Using the Joystick Select provides an on-screen Joystick to control two different parameters just like the Big Knobs do. To set up the Joystick, in the **Dial Parameters** section of the Inspector, specify which two parameters should be assigned to the X and Y axes of the Joystick.

Opening the **Joystick** section of the Inspector reveals the Joystick. Move the Joystick around and the corresponding Parameter will be changed. This will happen even if the Big Dial for that Parameter is not being displayed.

Attaching a Controller Instead of turning these Dials on the screen, you can turn them with a knob on your controller.

First, override the parameter in question as described above.

Second, in the **Dial Parameters** region of Select's Inspector, change the CC In device to the device corresponding to your controller.

Third, for each Dial you'd like controlled by the Controller, in the Dial Parameters region of Select's Inspector set the CC value of that Dial to the value you'll be sending from the Controller.

And you're done! Changing knobs on the controller will change the Dial, which in turn will update the parameters of the Children you bound to it.

5 Developing for Seq

Seq is written in pure Java, and is designed to have a simple model, even if the graphical interface code is relatively elaborate. The reason for this was that, ultimately, we hope Seq's model can be rearranged and customized by optimization and machine learning tools. For now, it's just an unusual sequencer.

5.1 Model and Dynamic Runtime Overview

The top-level class of Seq is `seq.engine.Seq`⁵ Seq is a singleton object which contains the entire sequence. It manages playing and recording the sequence, as well loading and saving it, and stores the sequence model.

⁵Well, that's not *quite* the case. There's a small bootstrapping class called `seq.Seq` so you can launch Seq as `java seq.Seq`.

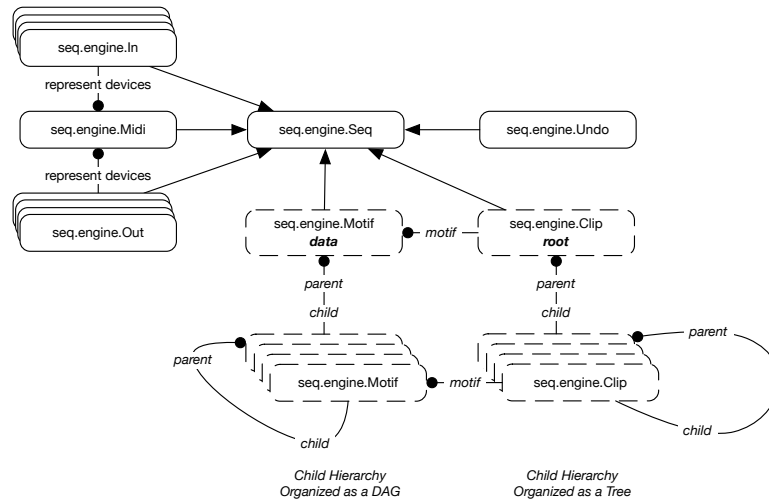


Figure 19 Model class relationships.

seq.engine.Seq also contains the global lock, which synchronizes between playing / recording notes and modifying / displaying the sequence from the GUI.

Seq runs the MIDI subsystem, both sending and receiving. The MIDI subsystem itself is managed by the class seq.engine.Midi, which Seq owns. Seq permits 16 outputs and 4 inputs. Each output and input represent a combination of a MIDI Device, a MIDI channel, and a nickname given by the user. These outputs and inputs are stored in 16 seq.engine.Out and 4 seq.engine.In objects respectively.

seq.engine.Seq maintains the sequence using two separate data structures. First, there is the **Motif DAG**, which stores the model itself. Second, there is the **Clip Tree**, which manages dynamically changing data while the model is playing.

Each of these graph structures contains an entry point where the sequence begins when you press play. For the DAG, it's a seq.engine.Motif object called **data**. For the Clip Tree, it's a corresponding seq.engine.Clip called **root**. This is called the root simply because it's the entry point for the sequence: though for the Clip Tree, it *is* the root of the tree, the corresponding **data** object isn't necessarily the root of the DAG at all (and there could be multiple roots).

The Motif DAG consists, at the bottom, of note-generating Motif objects such as Step Sequence or Notes. Every Motif may have multiple parent Motifs which indicate how to group their children together to play them, such as in series or in parallel. This DAG is loaded from and saved to a JSON structure using the seq.load() and seq.save() methods, which in turn call similar methods on each of the Motifs.

When the sequence is played, the seq.engine.Seq prepares and resets playing data, then starts the clock (a java.util.Timer⁶). The clock calls Seq.step() 192 times per beat (that is, 192 Parts Per Quarter-note, or PPQ) at the current BPM. step() acquires the lock, then calls advance() on the **root**, which in turn calls process on the root. This kicks off the series of recursive calls to advance and process throughout the root's Clip Tree, which gives each sim.engine.Clip in the tree a chance to advance itself forward one tick (1/192 PPQ). When this process is done, Seq lets go of the lock.

Each sim.engine.Clip is associated with a sim.engine.Motif in the model, and each will consult with its Motif to determine what notes to play, which children Clips to create and how to process them, and so on.. But there may be many Clips playing the same Motif at the same time, due to parallelism in play. Each of these Clips must maintain its own dynamic information (such as its current timestamp), so they have to be distinct from one another. To do this, the Clip must be a tree: parents cannot share the same child, they have to have

⁶java.util.Timer only has a resolution of 1ms. That's pretty bad, but it's what we have to work with. I can build a custom Timer using nanoTime rather than currentTimeMillis, but it only increases the resolution by about 1.5x. It's possible to build a timer using park and unpark, but it doesn't improve things much and is very computationally costly. The fundamental issue is the typical multitasking time-slice of the operating system.

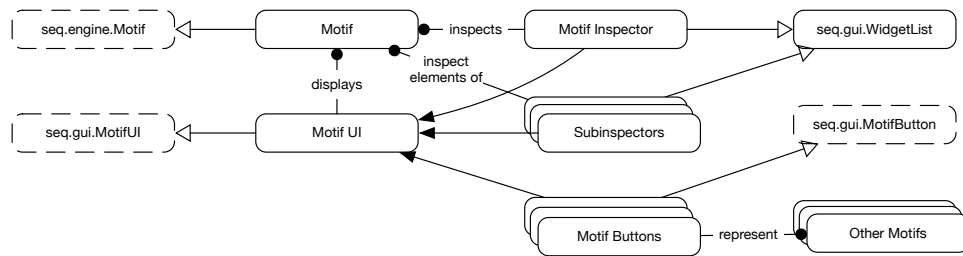


Figure 21 MotifUI relationships.

Many Motifs have child Motifs, and MotifUIs represent these child Motifs with drag-and-drop objects which are subclasses of `seq.gui.MotifButton`, just like `seq.gui.MotifListButton` was.

The GUI subsystem for a module *foo* would be stored in the Java package `seq.motif.foo.gui` and typically consist of:

- A MotifUI called `seq.motif.foo.gui.FooUI`, which interacts with the Motif and Clip, draws the Motif, and dictates which inspectors will be used.
- The primary inspector for the MotifUI, called `seq.motif.foo.gui.FooInspector`
- Any additional inspectors for subelements of the Motif
- If the Motif has children, a subclass of `MotifButton`, likely called `seq.motif.foo.gui.FooButton`, which displays those children.

5.3 Common GUI Widgets

To make your life easier, and to provide a consistent experience, Seq has a number of standard widgets which your MotifUI and inspectors may use:

seq.gui.WidgetList This is the superclass of inspectors, and is designed to make it easy to create labelled lists of JComponents. Typically all the JComponents, and all their labels, are provided in the constructor; or you can provide them in a later `build()` method.

seq.gui.SmallDial This is how Seq makes all of its dials. `SmallDial` has a range from 0.0 to 1.0, which you initialize up front. Then you override up to three methods. `setValue()` informs you that the user has just set the widget. Typically you'll acquire the lock (see Locking later) to update the model. `getValue()` asks you what value to display. Typically you'll acquire the lock (again, see Locking later) to read from the model. And `map()` asks how to print the data value next to the widget: by default this method formats its double value (0.0 to 1.0) as `x.xxxx`.

seq.gui.Dial A large version of `SmallDial`.

seq.gui.DisclosurePanel This is a JComponent which hides its children components until the user clicks on a ">", expanding it to display them. If you provide it with its parent component, it will inform the parent when it has expanded so the parent can revalidate itself.

seq.gui.PushButton A convenient class for constructing a `JButton` which performs an action, or which produces a pop-up menu and performs an action based on what is selected.

seq.gui.StringField A convenient class for editing text strings. When the user has edited a string, StringField will call the method `setValue()`, which you override, so you can update the Motif data or whatnot. Similarly to request the latest string from you to display, StringField will call `getValue()`. Like SmallDial, these methods will likely require you to acquire a lock to access the model within them.

seq.gui.TimeDisplay A rather convoluted collection of four SmallDials and a presets button which help the user to set a timestamp or a time interval. You override the methods `getTime()` to tell the TimeDisplay what time to display, and `setTime()` to update the model when the TimeDisplay is changed. These methods automatically acquire the lock, (see SmallDial), so you do not need to lock on the lock to manipulate the model in these methods. Additionally, the method `setTimeOutside()` allows you to update elements (such as the MotifUI) *outside* the lock.

seq.gui.Stretch A class for making stretchy invisible JComponents which fill all available horizontal or vertical space.

seq.gui.Collapse This transparent JComponent forces its child component to stay at its preferred size rather than expand to the (larger) size of the Collapse parent.

seq.gui.Dialogs Utility methods for creating dialog boxes.

seq.gui.SimpleColorMap A class for mapping real-numbered or integer values to colors.

seq.gui.ArgumentList Displays and edits the settings of arguments for child motifs.

seq.gui.DefaultParameterList Displays and edits the nicknames given to the Motif's parameters.

5.4 Building a Module

To build a module for Seq, you'll need to construct:

1. A Motif, which defines the parameters of the module and its relationship to child Motifs if any
2. A Clip, which performs the actions of the module when it is being played or recorded
3. a

5.5 Hints

Locking Whenever you modify the Motif from the MotifUI or one of its inspectors, you will need to acquire the global lock to prevent the model from attempting to play data while you are modifying it. Your goal is to acquire this lock, modify the data very rapidly, and then let go of the lock. You may even wish to acquire the lock multiple times in order to modify the data in a more fine-grained fashion. This is because the overriding goal is to keep Seq's playing accurate, even at the expense of GUI update. Seq needs to be able to acquire the lock to play the next step whenever it must.

Seq uses a **fair lock**. You may not be familiar with locks of this type. The standard Java lock, `synchronized`, is *unfair*. This means that the following starvation scenario can occur. Thread A acquires the lock. Thread B tries to acquire the lock but fails. It is blocked and put to sleep by the operating system. Thread A releases the lock. Because Thread B is asleep and can't ask for the lock, Thread A then decides to *reacquire* the lock again. Then Thread B wakes up and *still can't get the lock*. Thus Thread A can starve Thread B.

This actually happens quite a lot, especially in Linux, when dealing with GUI threads, and it is because on most operating systems (including MacOS, Windows, and Linux), the default lock mechanism is *unfair*: if a thread asks for a lock, there is no guarantee it will be the next in line to receive it (or ever!).

Java also provides *fair* locks via its `java.util.concurrent` package, but they are less convenient to use, and unlike `synchronized` you have to make sure to remember to *unlock* the lock. Fair locks are also significantly slower than unfair locks, which is why operating system locks are unfair by default. A fair lock is slow because when a thread fails to acquire the lock, it is put into a *queue* for that lock, then blocks. When a thread releases a lock, only the next thread in the queue is permitted to acquire the lock: all other requesting threads are stuck in the queue behind it. It is this queue that keeps things fair, but this extra machinery slows things down a bit. To be honest, even the high degree of locking and unlocking used in Seq is entirely inconsequential to performance.

You should lock like this:

```
import java.util.concurrent.*;

Seq seq = ...           // You need the Seq here
ReentrantLock lock = seq.getLock(); // This is Seq's global lock, and it is fair
lock.lock();             // acquire the lock
try
{
    // do your stuff
}
finally
{
    lock.unlock();       // always unlock in a finally so it's guaranteed to happen
}
```

You will see this pattern many *many* times in Seq: it's one of the things that makes Seq's GUI code so verbose! You do not ever need to lock in your Clip or your Motif: their methods are called while Seq's play thread has control of the lock.

You also don't need to lock when overriding the `TimeDisplay`'s `setTime()` or `getTime()`: they have already acquired the lock (in fact, to update the GUI *outside* the lock, `TimeDisplay` provides a special method called `setTimeOutside()`).